

ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



HaUI

SCHOOL OF INFORMATION
& COMMUNICATIONS TECHNOLOGY

BÁO CÁO THỰC NGHIỆM

Học phần: An ninh mạng

Chủ đề: Chuẩn DES và ứng dụng trong mã hóa dữ liệu (Sử dụng ngôn ngữ Java, Python)

Giáo viên hướng dẫn: TS. Phạm Văn Hiệp

Nhóm sinh viên thực hiện:

- | | |
|-------------------------|-------------------|
| 1. Chu Văn Ba | Mã SV: 2023605222 |
| 2. Hoàng Đức Trung Kiên | Mã SV: 2023605585 |
| 3. Nguyễn Như Lân | Mã SV: 2023607318 |
| 4. Nguyễn Kế Quang | Mã SV: 2023606562 |
| 5. Hà Minh Tú | Mã SV: 2023607110 |

Mã lớp học phần: 20252IT6070002

Nhóm: 06

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT.....	- 3 -
DANH MỤC HÌNH ẢNH.....	- 4 -
DANH MỤC BẢNG BIỂU	- 6 -
LỜI NÓI ĐẦU	- 7 -
CHƯƠNG 1: TỔNG QUAN VỀ AN NINH MẠNG VÀ THUẬT TOÁN DES	- 8 -
1.1. Tổng quan về An ninh mạng	- 8 -
1.1.1. Khái niệm và vai trò của an ninh mạng	- 8 -
1.1.2. Các nguyên tắc cơ bản trong bảo mật thông tin.....	- 9 -
1.1.3. Tổng quan về mã hóa dữ liệu và vai trò trong an ninh mạng ...	- 11 -
1.2. Các kiến thức cơ sở.....	- 13 -
1.2.1. Kiến thức toán học nền tảng	- 13 -
1.2.2. Các khái niệm cơ bản trong mật mã học.....	- 23 -
1.2.3. Giới thiệu ngôn ngữ lập trình sử dụng trong đề tài.....	- 23 -
1.3. Nội dung nghiên cứu.....	- 24 -
1.3.1. Lý do chọn đề tài.....	- 24 -
1.3.2. Nội dung nghiên cứu.....	- 24 -
CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU.....	- 26 -
2.1. Nghiên cứu, tìm hiểu hệ mã hóa khóa bí mật	- 26 -
2.1.1. Giới thiệu.....	- 26 -
2.1.2. Ứng dụng của hệ mã hóa khóa bí mật.....	- 26 -
2.1.3. Các phương pháp mã hóa khóa bí mật.....	- 28 -
2.1.4. Ưu điểm và nhược điểm.....	- 29 -
2.2. Chuẩn DES và ứng dụng trong thực tế	- 29 -
2.2.1. Lịch sử và giới thiệu chung về DES	- 29 -
2.2.2. Đặc điểm tổng quan của DES	- 30 -

2.2.3. Ứng dụng thực tế của DES.....	- 30 -
2.3. Nghiên cứu, tìm hiểu về chuẩn DES.....	- 30 -
2.3.1. Quá trình sinh khóa con	- 30 -
2.3.2. Quá trình mã hóa DES	- 33 -
2.3.3. Quá trình giải mã DES	- 37 -
2.4. Thiết kế chương trình, cài đặt thuật toán	- 38 -
2.4.1. Ngôn ngữ lập trình sử dụng	- 38 -
2.4.2. Cài đặt chương trình bằng Java.....	- 41 -
2.4.3. Cài đặt chương trình bằng Python	- 44 -
CHƯƠNG 3: KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM	- 53 -
3.1. Kiến thức kỹ năng đã học được trong quá trình thực hiện đề tài.....	- 53 -
3.2. Bài học kinh nghiệm	- 53 -
3.3. Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn...	- 54 -
TÀI LIỆU THAM KHẢO	- 55 -

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Từ hoàn chỉnh	Ý nghĩa
DES	Data Encryption Standard	Chuẩn mã hóa dữ liệu (Thuật toán mã hóa đối xứng kinh điển được nghiên cứu trong đề tài)
AES	Advanced Encryption Standard	Chuẩn mã hóa nâng cao (Thuật toán mã hóa hiện đại thay thế cho DES)
CIA	Confidentiality, Integrity, Availability	Bộ ba nguyên tắc cơ bản trong bảo mật thông tin: Tính bí mật, Tính toàn vẹn và Tính sẵn sàng
ISO	International Organization for Standardization	Tổ chức Tiêu chuẩn hóa Quốc tế
SSL	Secure Sockets Layer	Giao thức mã hóa giúp bảo mật truyền thông trên môi trường mạng
TLS	Transport Layer Security	Giao thức bảo mật tầng truyền tải (Bản kế thừa và phát triển an toàn hơn của SSL)
IP	Initial Permutation	Phép hoán vị khởi tạo đầu tiên trong quy trình mã hóa dữ liệu DES
IP^{-1}	Inverse Initial Permutation	Phép hoán vị khởi tạo nghịch đảo
PC1	Permuted Choice 1	Ma trận hoán vị/Lựa chọn hoán vị 1
PC2	Permuted Choice 2	Ma trận hoán vị/Lựa chọn hoán vị 2
LS	Left Shift	Phép dịch bit vòng sang bên trái
S-box	Substitution-box	Hộp thay thế
P-box	Permutation-box	Hộp hoán vị
GUI	Graphical User Interface	Giao diện đồ họa người dùng
XOR	Exclusive OR	Phép toán logic "Hoặc loại trừ"
UTF-8	Unicode Transformation Format – 8-bit	Định dạng mã hóa ký tự Unicode sử dụng trong chương trình để hỗ trợ hiển thị dữ liệu tiếng Việt đa byte

DANH MỤC HÌNH ẢNH

Hình 1.1: Sơ đồ tổng quan các thành phần của an ninh mạng.....	- 9 -
Hình 1.2: Tam giác CIA	- 9 -
Hình 1.3: Sơ đồ quá trình mã hóa và giải mã dữ liệu	- 11 -
Hình 1.4: Mã hóa đối xứng	- 11 -
Hình 1.5: Mã hóa bất đối xứng	- 12 -
Hình 1.6: So sánh tổng quan giữa mã hóa đối xứng và bất đối xứng.....	- 13 -
Hình 1.7: Sơ đồ minh họa tính chất nghịch đảo của XOR trong quá trình mã hóa và giải mã	- 15 -
Hình 1.8: Bảng IP	- 15 -
Hình 1.9: Bảng IP^{-1}	- 15 -
Hình 1.10: Cách hoạt động của một vòng Feistel.....	- 16 -
Hình 1.11: Sơ đồ quá trình sinh khóa con.....	- 18 -
Hình 1.12: Bảng hoán vị PC1	- 18 -
Hình 1.13: Bảng số lần dịch vòng.....	- 18 -
Hình 1.14: Bảng hoán vị PC2	- 18 -
Hình 1.15: Hàm F.....	- 19 -
Hình 1.16: Bảng hoán vị E.....	- 20 -
Hình 1.17: S-box 1	- 20 -
Hình 1.18: S-box 2	- 21 -
Hình 1.19: S-box 3	- 21 -
Hình 1.20: S-box 4	- 21 -
Hình 1.21: S-box 5	- 21 -
Hình 1.22: S-box 6	- 22 -
Hình 1.23: S-box 7	- 22 -
Hình 1.24: S-box 8	- 22 -
Hình 1.25: P-box	- 22 -
Hình 2.1: Sơ đồ khối của hệ truyền tin mật	- 26 -

Hình 2.2: Ví dụ minh họa tính xác thực trong mã hóa khóa bí mật.....	- 27 -
Hình 2.3: Sơ đồ trao đổi khóa bí mật giữa nhiều người dùng	- 27 -
Hình 2.4: Sơ đồ phân phối khóa qua KDC	- 28 -
Hình 2.5: Sơ đồ quá trình sinh khóa con.....	- 31 -
Hình 2.6: Sơ đồ mã hóa DES tổng quát.....	- 34 -
Hình 2.7: Chi tiết một vòng lặp DES	- 35 -
Hình 2.8: Minh họa mã hóa thành công.....	- 43 -
Hình 2.9: Minh họa lưu file bản mã.....	- 43 -
Hình 2.10: Minh họa mở file bản mã.....	- 44 -
Hình 2.11: Minh họa giải mã thành công.....	- 44 -
Hình 2.12: Giao diện tổng thể chương trình DES Python	- 48 -
Hình 2.13: Kết quả kịch bản 1	- 49 -
Hình 2.14: Kết quả kịch bản 2	- 50 -
Hình 2.15: Kết quả kịch bản 3	- 51 -
Hình 2.16: Kết quả kịch bản 4	- 52 -

DANH MỤC BẢNG BIỂU

Bảng 1.1. Bảng minh họa quá trình chuyển đổi một chuỗi ký tự sang dạng nhị phân	- 13 -
Bảng 1.2. Bảng phép toán XOR cơ bản	- 14 -
Bảng 2.1: Bảng Hoán vị PC1	- 31 -
Bảng 2.2: Bảng giá trị C_i , D_i qua 16 vòng.....	- 32 -
Bảng 2.3: Bảng hoán vị PC2	- 33 -
Bảng 2.4: Bảng hoán vị IP	- 34 -
Bảng 2.5: Bảng 16 vòng lặp L_i , R_i	- 35 -
Bảng 2.6: Bảng mở rộng E.....	- 36 -
Bảng 2.7: Bảng hoán vị P.....	- 36 -
Bảng 2.8: Bảng hoán vị IP^{-1}	- 37 -

LỜI NÓI ĐẦU

Trong thời đại công nghệ số phát triển mạnh mẽ, bảo vệ thông tin và dữ liệu ngày càng trở nên cấp thiết. Các hành vi tấn công mạng, đánh cắp dữ liệu và giả mạo thông tin ngày một tinh vi, đòi hỏi các tổ chức và cá nhân phải áp dụng các biện pháp bảo mật hiệu quả, trong đó mã hóa dữ liệu đóng vai trò then chốt.

DES (Data Encryption Standard) là một trong những thuật toán mã hóa đối xứng kinh điển, từng được công nhận là chuẩn mã hóa quốc gia tại Hoa Kỳ và ứng dụng rộng rãi trong nhiều thập kỷ. Dù hiện nay đã được thay thế bởi các thuật toán hiện đại hơn như AES, DES vẫn là nền tảng lý tưởng để học tập và nghiên cứu mật mã học nhờ cấu trúc Feistel rõ ràng, các bước hoán vị, thay thế và sinh khóa được thiết kế chặt chẽ.

Nhóm chúng em lựa chọn đề tài "*Chuẩn DES và ứng dụng trong mã hóa dữ liệu sử dụng Java, Python*" với mục tiêu nghiên cứu lý thuyết và triển khai thực tế thuật toán DES thông qua hai ngôn ngữ lập trình phổ biến hiện nay. Qua đó, nhóm không chỉ củng cố kiến thức về an ninh thông tin mà còn rèn luyện kỹ năng lập trình, tư duy thuật toán và làm việc nhóm.

Báo cáo được chia thành ba chương:

Chương 1: Tổng quan về an ninh mạng và thuật toán DES — trình bày tổng quan về an ninh mạng, các kiến thức cơ sở và nội dung nghiên cứu.

Chương 2: Kết quả nghiên cứu — trình bày chi tiết quá trình cài đặt thuật toán trên Java và Python, giao diện chương trình và kết quả kiểm thử.

Chương 3: Kết luận và bài học kinh nghiệm — tổng kết kiến thức thu được, đánh giá ưu nhược điểm của DES và chia sẻ kinh nghiệm thực tiễn.

Nhóm hy vọng báo cáo mang lại góc nhìn rõ ràng và thực tiễn về thuật toán DES, góp phần nâng cao nhận thức về tầm quan trọng của bảo mật dữ liệu trong thời đại số.

CHƯƠNG 1: TỔNG QUAN VỀ AN NINH MẠNG VÀ THUẬT TOÁN DES

1.1. Tổng quan về An ninh mạng

Trong thời đại công nghệ số phát triển vượt bậc, thông tin đã trở thành một tài sản có giá trị đặc biệt quan trọng đối với cá nhân, tổ chức và quốc gia. Sự bùng nổ của Internet và các thiết bị kết nối đã mang lại vô số tiện ích trong cuộc sống, nhưng đồng thời cũng kéo theo những rủi ro nghiêm trọng về bảo mật. Các cuộc tấn công mạng ngày càng tinh vi, có tổ chức và gây ra thiệt hại lớn về kinh tế lẫn uy tín cho các tổ chức trên toàn thế giới. Chính vì vậy, an ninh mạng đã trở thành một lĩnh vực không thể thiếu trong sự phát triển của công nghệ thông tin hiện đại.

1.1.1. Khái niệm và vai trò của an ninh mạng

An ninh mạng (Cybersecurity) là tập hợp các biện pháp, công nghệ và quy trình được thiết kế nhằm bảo vệ các hệ thống máy tính, mạng lưới, chương trình và dữ liệu khỏi các cuộc tấn công, truy cập trái phép, hư hỏng hoặc các mối đe dọa khác từ môi trường số. Nói một cách đơn giản hơn, an ninh mạng là việc đảm bảo rằng thông tin và hệ thống luôn được bảo vệ một cách toàn vẹn, bí mật và sẵn sàng hoạt động.

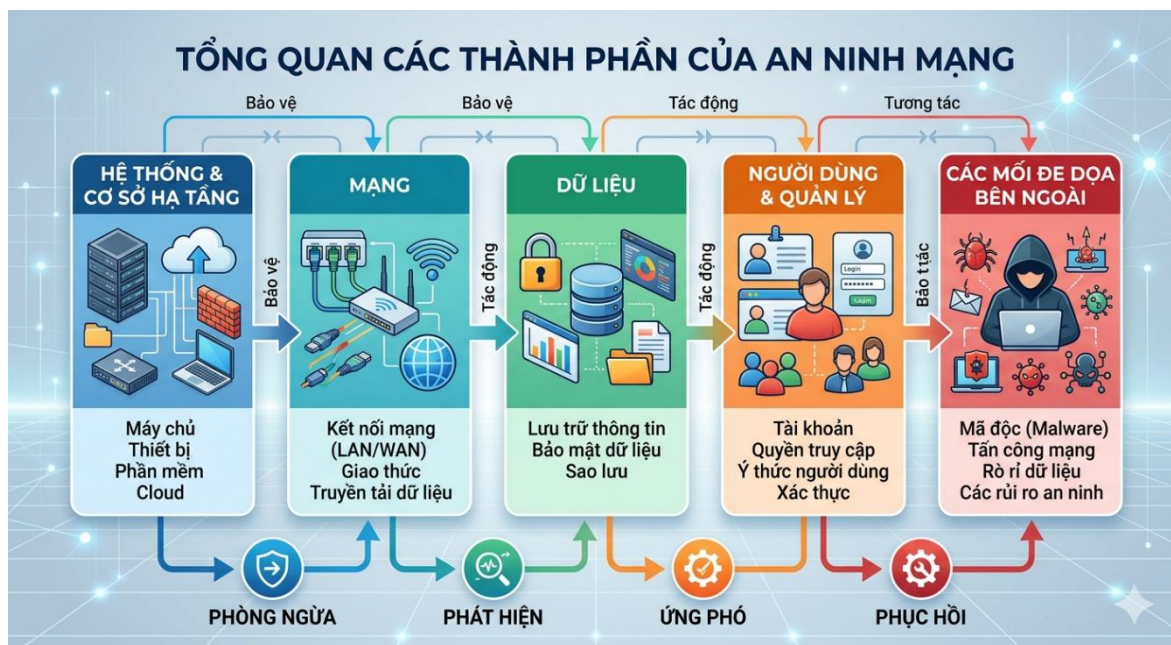
Theo Tổ chức Tiêu chuẩn hóa Quốc tế (ISO), an ninh thông tin được định nghĩa là việc bảo vệ thông tin và các hệ thống thông tin khỏi sự truy cập, sử dụng, tiết lộ, gián đoạn, sửa đổi hoặc phá hủy trái phép, nhằm đảm bảo tính bí mật, toàn vẹn và tính sẵn sàng của thông tin.

An ninh mạng đóng vai trò then chốt trong nhiều lĩnh vực của đời sống hiện đại:

- **Bảo vệ dữ liệu cá nhân:** Ngăn chặn việc thông tin nhạy cảm của người dùng như số tài khoản ngân hàng, thông tin y tế, dữ liệu định danh bị đánh cắp hoặc lạm dụng.
- **Đảm bảo hoạt động liên tục của tổ chức:** Các doanh nghiệp, cơ quan nhà nước cần hệ thống mạng hoạt động ổn định, không bị gián đoạn bởi các cuộc tấn công để duy trì hoạt động sản xuất, kinh doanh và cung cấp dịch vụ.
- **Bảo vệ cơ sở hạ tầng quốc gia:** Các hệ thống điện, nước, giao thông, tài chính đều được vận hành qua mạng máy tính. Một cuộc tấn công

vào các hệ thống này có thể gây ra hậu quả nghiêm trọng ở quy mô quốc gia.

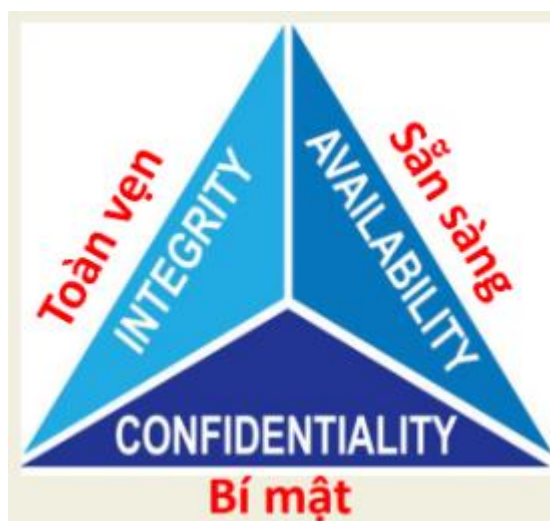
- **Xây dựng niềm tin trong giao dịch số:** An ninh mạng là nền tảng để người dùng tin tưởng vào các giao dịch thương mại điện tử, ngân hàng trực tuyến và các dịch vụ số khác.



Hình 1.1: Sơ đồ tổng quan các thành phần của an ninh mạng

1.1.2. Các nguyên tắc cơ bản trong bảo mật thông tin

Nền tảng lý thuyết của an ninh mạng được xây dựng dựa trên ba nguyên tắc cốt lõi, thường được gọi là mô hình CIA Triad (hay tam giác bảo mật CIA), bao gồm: Tính bí mật (Confidentiality), Tính toàn vẹn (Integrity) và Tính sẵn sàng (Availability).



Hình 1.2: Tam giác CIA

a) Tính bí mật (Confidentiality)

Tính bí mật đảm bảo rằng thông tin chỉ được phép truy cập bởi những cá nhân, hệ thống hoặc tiến trình được ủy quyền hợp lệ. Bất kỳ sự tiết lộ thông tin nào ra ngoài phạm vi cho phép đều được xem là vi phạm tính bí mật.

Để đảm bảo tính bí mật, các biện pháp phổ biến bao gồm:

- Mã hóa dữ liệu trong quá trình lưu trữ và truyền tải (sử dụng các chuẩn như DES, AES, RSA).
- Kiểm soát quyền truy cập thông qua cơ chế xác thực người dùng.
- Sử dụng các giao thức bảo mật như SSL/TLS trong truyền thông mạng.

b) Tính toàn vẹn (Integrity)

Tính toàn vẹn đảm bảo rằng thông tin không bị thay đổi, chỉnh sửa hoặc xóa một cách trái phép trong suốt quá trình lưu trữ, xử lý và truyền tải. Nói cách khác, dữ liệu nhận được phải hoàn toàn giống với dữ liệu ban đầu được gửi đi.

Các kỹ thuật thường dùng để bảo vệ tính toàn vẹn:

- Hàm băm (Hash Function) như MD5, SHA-256 để phát hiện sự thay đổi dữ liệu.
- Chữ ký số (Digital Signature) để xác minh nguồn gốc và tính nguyên vẹn của thông tin.
- Cơ chế checksum trong truyền thông mạng.

c) Tính sẵn sàng (Availability)

Tính sẵn sàng đảm bảo rằng các hệ thống, dịch vụ và dữ liệu luôn có thể truy cập và sử dụng được bởi người dùng hợp lệ vào bất kỳ thời điểm nào khi có nhu cầu. Đây là yếu tố quan trọng đặc biệt đối với các hệ thống phục vụ liên tục như ngân hàng, y tế, giao thông.

Các biện pháp đảm bảo tính sẵn sàng bao gồm:

- Xây dựng hệ thống dự phòng và sao lưu dữ liệu định kỳ.
- Triển khai hệ thống phòng chống tấn công từ chối dịch vụ (Anti-DDoS).
- Sử dụng bộ cân bằng tải (Load Balancer) để phân phối lưu lượng truy cập.

Ba nguyên tắc trên có mối quan hệ chặt chẽ và bổ trợ cho nhau. Một hệ thống bảo mật hiệu quả cần đảm bảo cả ba yếu tố này đồng thời. Việc xem nhẹ

bất kỳ nguyên tắc nào đều có thể dẫn đến những lỗ hổng nghiêm trọng trong hệ thống bảo mật.

1.1.3. Tổng quan về mã hóa dữ liệu và vai trò trong an ninh mạng

Mã hóa dữ liệu (Data Encryption) là quá trình chuyển đổi thông tin từ dạng có thể đọc được (bản rõ - plaintext) sang dạng không thể đọc được nếu không có khóa giải mã tương ứng (bản mã - ciphertext). Đây là một trong những công cụ mạnh mẽ và hiệu quả nhất để bảo vệ thông tin trong thế giới số.



Hình 1.3: Sơ đồ quá trình mã hóa và giải mã dữ liệu

Dựa trên cơ chế sử dụng khóa, mã hóa được chia thành hai loại chính:

- **Mã hóa đối xứng (Symmetric Encryption):** Sử dụng cùng một khóa cho cả quá trình mã hóa và giải mã. Ưu điểm là tốc độ nhanh, phù hợp mã hóa lượng dữ liệu lớn. Nhược điểm là việc trao đổi và quản lý khóa phức tạp. Các thuật toán tiêu biểu: DES, 3DES, AES.



Hình 1.4: Mã hóa đối xứng

- **Mã hóa bất đối xứng (Asymmetric Encryption):** Sử dụng cặp khóa công khai (public key) và khóa bí mật (private key). Khóa công khai dùng để mã hóa, khóa bí mật dùng để giải mã. Ưu điểm là giải quyết được vấn đề phân phối khóa, nhưng tốc độ xử lý chậm hơn. Các thuật toán tiêu biểu: RSA, ElGamal, ECC.

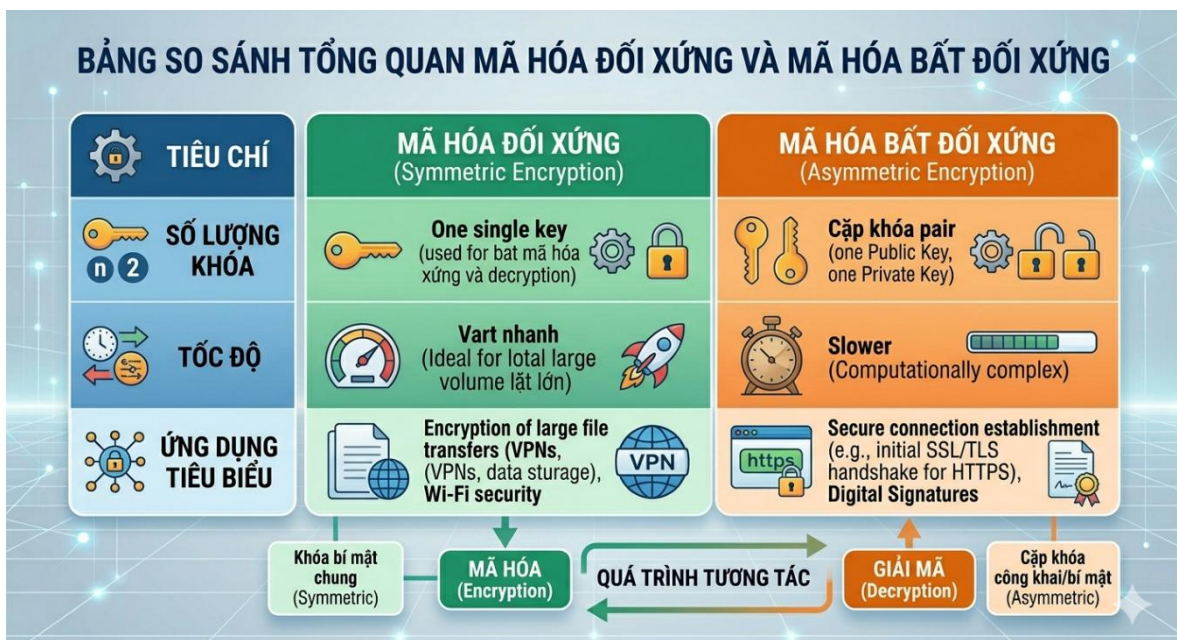


Hình 1.5: Mã hóa bất đối xứng

Mã hóa đóng vai trò trung tâm trong việc đảm bảo an ninh mạng thông qua các chức năng sau:

- **Bảo vệ dữ liệu truyền tải:** Thông tin được mã hóa trước khi truyền qua mạng, đảm bảo rằng dù bị chặn bắt, kẻ tấn công cũng không thể đọc được nội dung.
- **Bảo vệ dữ liệu lưu trữ:** Các tệp, cơ sở dữ liệu được mã hóa để ngăn chặn truy cập trái phép ngay cả khi thiết bị lưu trữ bị đánh cắp.
- **Xác thực danh tính:** Kết hợp với chữ ký số, mã hóa giúp xác minh danh tính của các bên tham gia giao tiếp, ngăn chặn giả mạo.
- **Đảm bảo tính toàn vẹn của dữ liệu:** Các kỹ thuật mã hóa được sử dụng trong hàm băm để phát hiện bất kỳ sự thay đổi nào đối với dữ liệu gốc.

Trong bối cảnh đó, chuẩn mã hóa DES (Data Encryption Standard) — đối tượng nghiên cứu chính của đề tài này — là một trong những thuật toán mã hóa đối xứng kinh điển, đặt nền móng quan trọng cho sự phát triển của mật mã học hiện đại. Mặc dù đã được thay thế bởi AES trong thực tiễn, DES vẫn có giá trị học thuật to lớn và là bước đệm cần thiết để hiểu các hệ thống mã hóa phức tạp hơn.



Hình 1.6: So sánh tổng quan giữa mã hóa đối xứng và bất đối xứng

1.2. Các kiến thức cơ sở

Để hiểu rõ nguyên lý hoạt động của thuật toán DES, chúng ta cần nắm vững một số kiến thức nền tảng về toán học, mật mã học và lập trình. Phần này trình bày các khái niệm cơ bản làm cơ sở lý thuyết cho việc phân tích và cài đặt thuật toán DES ở các phần tiếp theo.

1.2.1. Kiến thức toán học nền tảng

1.2.1.1. Hệ nhị phân và biểu diễn dữ liệu

Máy tính xử lý và lưu trữ tất cả thông tin dưới dạng các con số nhị phân, tức là dãy các bit có giá trị 0 hoặc 1. Đây là nền tảng cơ bản để hiểu cách thuật toán DES hoạt động, vì toàn bộ quá trình mã hóa của DES được thực hiện trên các chuỗi bit.

Hệ nhị phân là hệ đếm cơ số 2, chỉ sử dụng hai ký hiệu là 0 và 1. Mỗi chữ số trong hệ nhị phân được gọi là một bit (binary digit). Nhóm 8 bit liên tiếp tạo thành một byte.

Ví dụ về chuyển đổi giữa các hệ:

- Số thập phân 65 → Nhị phân: **01000001**
- Ký tự 'A' có mã ASCII = 65 → Nhị phân: **01000001**
- Số thập lục phân (Hex) 4F → Nhị phân: **01001111**

DES xử lý dữ liệu theo từng khối 64 bit (tương đương 8 byte). Trước khi thực hiện mã hóa, mọi dạng dữ liệu đầu vào (văn bản, tệp tin,...) đều phải được chuyển đổi sang chuỗi bit nhị phân tương ứng. Khóa mã hóa cũng có độ dài 64 bit, trong đó 56 bit được sử dụng thực sự cho quá trình mã hóa, 8 bit còn lại dùng để kiểm tra lỗi.

Bảng 1.1. Bảng minh họa quá trình chuyển đổi một chuỗi ký tự sang dạng nhị phân

Ký tự gốc	Mã thập phân(ASCII)	Công thức tính nhị phân	Dãy nhị phân
D	68	$64 + 4 = 2^6 + 2^2$	01000100
E	69	$64 + 4 + 1 = 2^6 + 2^2 + 2^0$	01000101
S	83	$64 + 16 + 2 + 1 = 2^6 + 2^4 + 2^1 + 2^0$	01010011

1.2.1.2. Phép toán XOR và ứng dụng trong mã hóa

Phép toán XOR (Exclusive OR — hay còn gọi là phép hoặc loại trừ) là một trong những phép toán logic quan trọng nhất trong mật mã học, được sử dụng xuyên suốt trong thuật toán DES.

Định nghĩa: XOR là phép toán logic trên hai bit, cho kết quả là 1 nếu hai bit đầu vào khác nhau, và cho kết quả là 0 nếu hai bit đầu vào giống nhau.

Bảng 1.2. Bảng phép toán XOR cơ bản

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XOR có một tính chất đặc biệt hữu ích trong mật mã học, đó là tính nghịch đảo chính nó:

Nếu: $A \oplus B = C$

Thì: $C \oplus B = A$

Tính chất này có nghĩa là nếu dùng khóa K để mã hóa bản rõ P thành bản mã C bằng phép XOR, thì cũng dùng chính khóa K đó để giải mã C trở về P. Đây là lý do XOR được ứng dụng rộng rãi trong các thuật toán mã hóa đối xứng như DES.

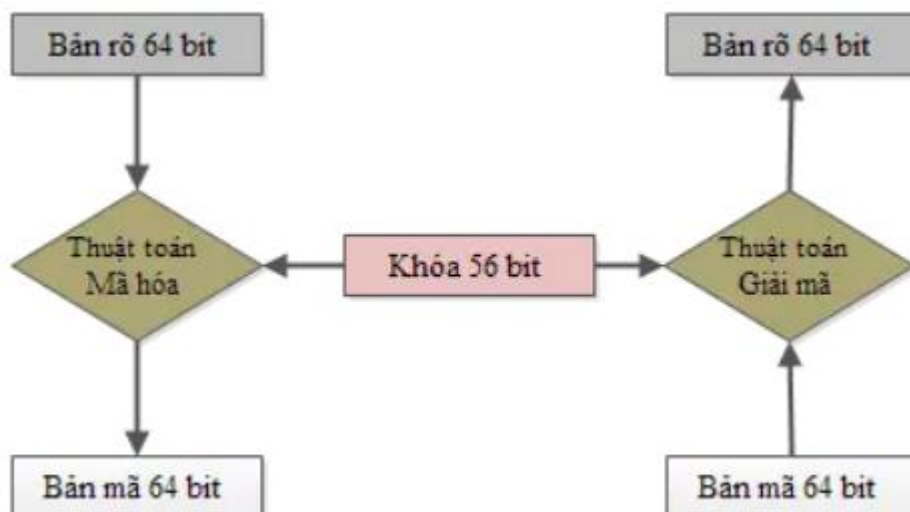
Ví dụ minh họa:

Bản rõ P = **1011 0110**

Khóa K = **1100 1010**

Bản mã C = $P \oplus K = \mathbf{0111\ 1100}$

Giải mã: $C \oplus K = \mathbf{1011\ 0110} = P$



Hình 1.7: Sơ đồ minh họa tính chất nghịch đảo của XOR trong quá trình mã hóa và giải mã

1.2.1.3. Bảng hoán vị

a, Hoán vị khởi tạo – IP

Hoán vị là thay đổi vị trí các bit trong một chuỗi giá trị nhưng không làm thay đổi giá trị của các bit này. Đây là bước đầu tiên trong quy trình mã hóa dữ liệu. 64 bit dữ liệu đầu vào, gọi là plaintext, sẽ được hoán vị theo bảng mô tả sau đây.

IP =	58	50	42	34	26	18	10	2
	60	52	44	36	28	20	12	4
	62	54	46	38	30	22	14	6
	64	56	48	40	32	24	16	8
	57	49	41	33	25	17	9	1
	59	51	43	35	27	19	11	3
	61	53	45	37	29	21	13	5
	63	55	47	39	31	23	15	7

Hình 1.8: Bảng IP

Chuỗi bit đầu vào được đánh số từ 1 đến 64. Sau đó, các bit này được thay đổi vị trí như sơ đồ IP, bit số 58 được đặt vào vị trí đầu tiên, bit số 50 được đặt vào vị trí thứ 2. Cứ như vậy, bit thứ 7 được đặt vào vị trí cuối cùng.

b, Bảng hoán vị IP^{-1}

- IP^{-1} là bước hoán vị cuối cùng trong DES, ngược lại với IP
- Áp dụng sau khi 16 vòng Feistel hoàn tất và kết hợp R16 || L16.
- IP^{-1} trả kết quả về đúng thứ tự ban đầu của bit trước IP.

$IP^{-1} =$	40	8	48	16	56	24	64	32
	39	7	47	15	55	23	63	31
	38	6	46	14	54	22	62	30
	37	5	45	13	53	21	61	29
	36	4	44	12	52	20	60	28
	35	3	43	11	51	19	59	27
	34	2	42	10	50	18	58	26
	33	1	41	9	49	17	57	25

Hình 1.9: Bảng IP^{-1}

1.2.1.4. Cấu trúc vòng Feistel

a, Khái niệm

Cấu trúc Feistel là một mô hình thiết kế thuật toán mã hóa khối. Cấu trúc này chia dữ liệu thành hai phần và thực hiện các phép biến đổi lặp lại nhiều vòng dựa trên một hàm và khóa con (DES sử dụng cấu trúc Feistel với 16 vòng).

b, Cách hoạt động của một vòng Feistel

Giả sử đầu vào là khối dữ liệu 64 bit được chia thành hai phần:

L_0 (32 bit trái)

R_0 (32 bit phải)

Trong mỗi vòng i , thực hiện các bước sau:

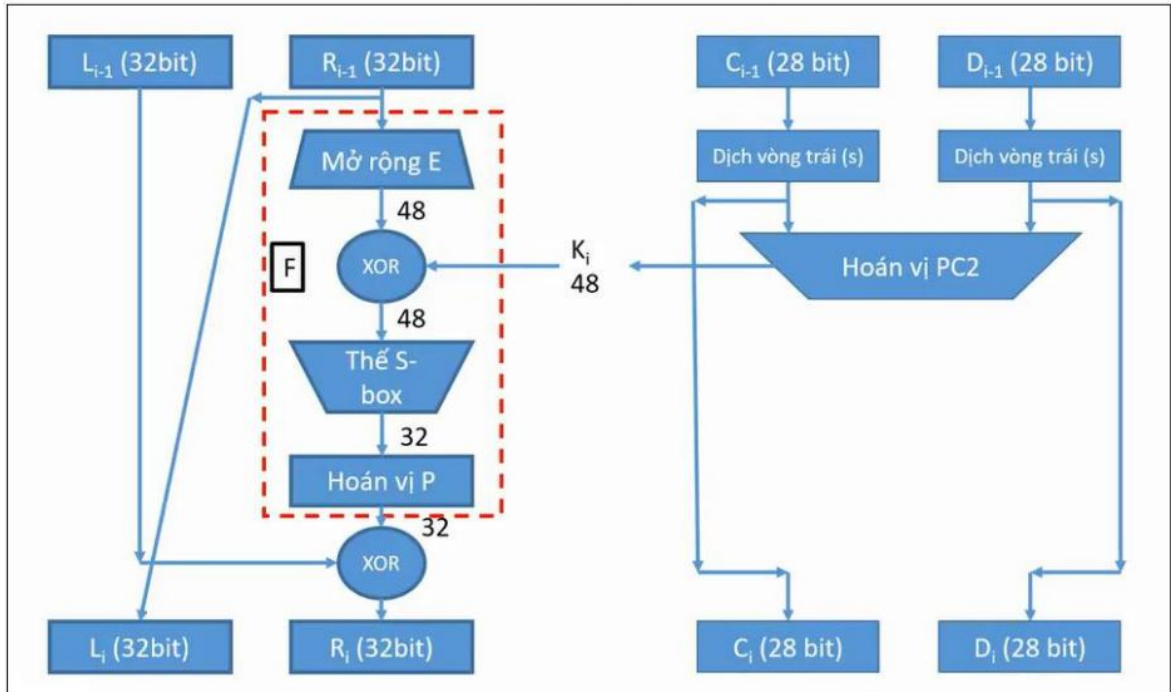
$$\begin{cases} L_i = R_{i-1} \\ R_i = L_{i-1} \oplus F(R_{i-1}, K_i) \end{cases}$$

Trong đó:

- F là hàm vòng, thực hiện trên nửa phải và khóa con K_i

- $\oplus$ là phép XOR

Cấu trúc Feistel được thể hiện trong hình minh họa sau đây:



Hình 1.10: Cách hoạt động của một vòng Feistel

Khối đầu vào của mỗi vòng được chia thành hai nửa có thể được ký hiệu là L và R cho nửa bên trái và nửa bên phải.

Trong mỗi vòng, nửa bên phải của khối, R, không thay đổi. Nhưng nửa bên trái, L, trải qua một phép toán phụ thuộc vào R và khóa mã hóa. Đầu tiên, chúng ta áp dụng một hàm mã hóa f lấy hai đầu vào – khóa K và R. Hàm tạo ra đầu ra $f(R,K)$. Sau đó, chúng ta XOR đầu ra của hàm toán học với L.

Trong quá trình triển khai thực tế của Feistel Cipher, chẳng hạn như DES, thay vì sử dụng toàn bộ khóa mã hóa trong mỗi vòng, một khóa phụ thuộc vào vòng (khóa phụ) được lấy từ khóa mã hóa. Điều này có nghĩa là mỗi vòng sử dụng một khóa khác nhau, mặc dù tất cả các khóa phụ này đều liên quan đến khóa gốc.

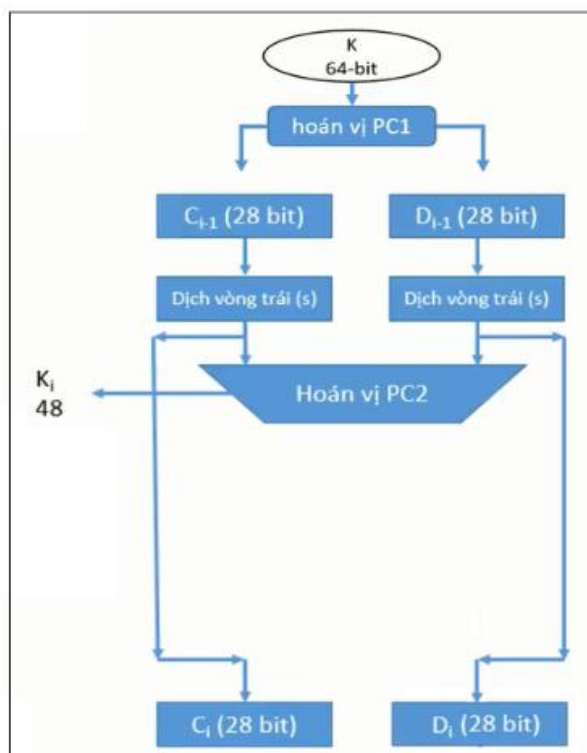
Bước hoán vị ở cuối mỗi vòng hoán đổi L đã sửa đổi và R chưa sửa đổi. Do đó, L cho vòng tiếp theo sẽ là R của vòng hiện tại. Và R cho vòng tiếp theo sẽ là đầu ra L của vòng hiện tại.

Các bước thay thế và hoán vị trên tạo thành một vòng. Số vòng được chỉ định bởi thiết kế thuật toán.

Khi vòng cuối cùng hoàn tất thì hai khối con R và L được nối theo thứ tự này để tạo thành khối văn bản mã hóa.

1.2.1.5. Quá trình sinh khóa con

16 vòng lặp của DES chạy cùng thuật toán như nhau nhưng với 16 khóa con khác nhau. Các khóa con đều được sinh ra từ khóa chính của DES bằng thuật toán sinh khóa con.



Hình 1.11: Sơ đồ quá trình sinh khóa con

Khóa ban đầu là 1 chuỗi có độ dài 64 bit, bit thứ 8 của mỗi byte sẽ được lấy ra để kiểm tra phát hiện lỗi, tạo ra chuỗi 56 bit. Sau khi bỏ các bit kiểm tra ta sẽ hoán vị chuỗi 56 bit này. Hai bước trên được thực hiện thông qua hoán vị ma trận PC1.

PC1 =	57	49	41	33	25	17	9	1
	58	50	42	34	26	18	10	2
	59	51	43	35	27	19	11	3
	60	52	44	36	63	55	47	39
	31	23	15	7	62	54	46	38
	30	22	14	6	61	53	45	37
	29	21	13	5	28	20	12	4

Hình 1.12: Bảng hoán vị PC1

Tiếp theo ta kết quả sau khi PC1 thành 2 phần : C0 : 28 bit đầu. D0 : 28 bit cuối. Mỗi phần sẽ được xử lý 1 cách độc lập. $C_i = \text{LSi}(C_{i-1})$; $D_i = \text{LSi}(D_{i-1})$ với $1 \leq i \leq 16$. LSi là biểu diễn phép dịch bit vòng (cyclic shift) sang trái 1 hoặc 2 vị trí tùy thuộc vào i.

Vòng lặp	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Số lần dịch trái	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Hình 1.13: Bảng số lần dịch vòng

Cuối cùng sử dụng hoán vị cố định PC2 để hoán vị chuỗi $C_i D_i$ 56 bit tạo thành khóa K_i với 48 bit.

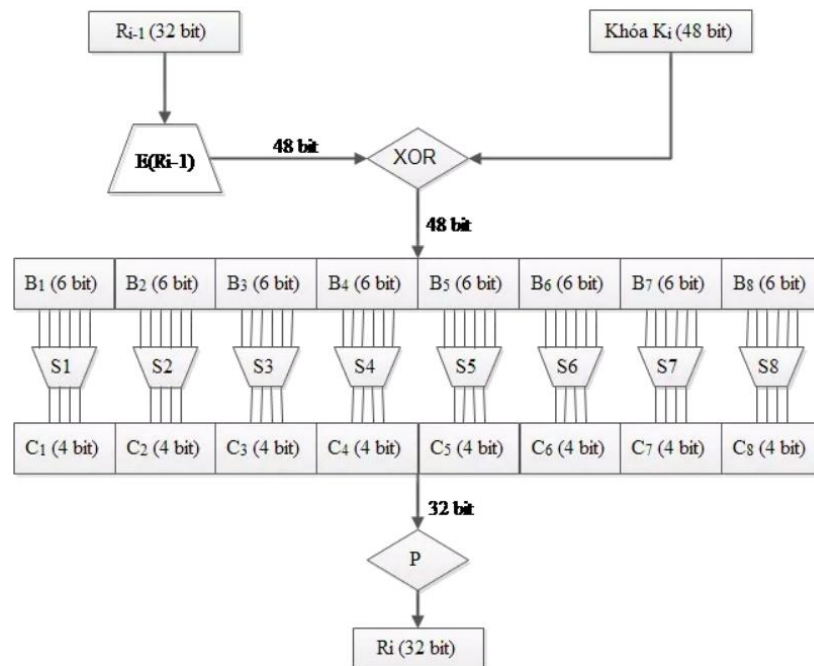
PC2 =	14	17	11	24	1	5
	3	28	15	6	21	10
	23	19	12	4	26	8
	16	7	27	20	13	2
	41	52	31	37	47	55
	30	40	51	45	33	48
	44	49	39	56	34	53
	46	42	50	36	29	32
Có 48 bit						

Hình 1.14: Bảng hoán vị PC2

1.2.1.6. Hàm F

Đầu vào hàm f có 2 biến: Biến thứ nhất: R_{i-1} là xâu bit có độ dài 32 bit. Biến thứ hai: K_i là xâu bit có độ dài 48 bit. Đầu ra của hàm f là xâu có độ dài 32 bit. Quy trình hoạt động của hàm f như sau:

- Biến thứ nhất R_{i-1} được mở rộng thành một xâu có độ dài 48 bit theo một hàm mở rộng hoán vị E (Expansion permutation). Thực chất hàm mở rộng $E(R_{i-1})$ là một hoán vị có lặp trong đó lặp lại 16 bit của R_{i-1} .
- Tính $E(R_{i-1}) \text{ XOR } K_i$.
- Tách kết quả của phép tính trên thành 8 xâu 6 bit $B_1, B_2, \dots, B_8$.
- Đưa các khối 8 bit B_i vào 8 bảng $S_1, S_2, \dots, S_8$ (được gọi là các hộp S-box). Mỗi hộp S-Box là một bảng 4×16 cố định có các cột từ 0 đến 15 và các hàng từ 0 đến 3. Với mỗi xâu 6 bit $B_i = b_1b_2b_3b_4b_5b_6$ ta tính được $S_i(B_i)$ như sau: hai bit b_1b_6 xác định hàng r trong hộp S_i , bốn bit $b_2b_3b_4b_5$ xác định cột c trong hộp S_i . Khi đó, $S_i(B_i)$ sẽ xác định phần tử $C_i = S_i(r, c)$, phần tử này viết dưới dạng nhị phân 4 bit. Như vậy, 8 khối 6 bit B_i ($1 \leq i \leq 8$) sẽ cho ra 8 khối 4 bit C_i với ($1 \leq i \leq 8$).
- Xâu bit $C = C_1C_2C_3C_4C_5C_6C_7C_8$ có độ dài 32 bit được hoán vị theo phép toán hoán vị P . Kết quả $P(C)$ sẽ là kết quả của hàm $f(R_{i-1}, K_i)$.



Hình 1.15: Hàm F

a, Hàm mở rộng E

Hàm mở rộng E sẽ tăng độ dài R_{i-1} từ 32 bit lên 48 bit bằng cách thay đổi thứ tự các bit cũng như lặp lại các bit. Việc thực hiện này nhằm hai mục đích:

- Làm độ dài của R_{i-1} cùng cỡ với khóa K để thực hiện việc cộng modulo XOR.
- Cho kết quả dài hơn để có thể được nén trong suốt quá trình thay thế. Tuy nhiên, cả hai mục đích này nhằm một mục tiêu chính là bảo mật dữ liệu. Bằng cách cho phép 1 bit có thể chèn vào hai vị trí thay thế, sự phụ thuộc của các bit đầu ra với các bit đầu vào sẽ trải rộng ra.

E =	32	1	2	3	4	5
	4	5	6	7	8	9
	8	9	10	11	12	13
	12	13	14	15	16	17
	16	17	18	19	20	21
	20	21	22	23	24	25
	24	25	26	27	28	29
	28	29	30	31	32	1

Hình 1.16: Bảng hoán vị E

b, Các hộp S-box

Sau khi thực hiện phép XOR giữa $E(R_{i-1})$ và K_i , kết quả thu được chuỗi 48 bit chia làm 8 khối đưa vào 8 hộp S-box. Mỗi hộp S-Box sẽ có 6 bit đầu vào và 4 bit đầu ra. Kết quả thu được là một chuỗi 32 bit tiếp tục vào hộp P-Box.

- Mỗi hàng trong mỗi hộp S là hoán vị của các số nguyên từ 0 đến 15.
- Các hộp S-box phi tuyến tính. Nói cách khác, đầu ra không phải là biến đổi tuyến tính của đầu vào.
- Sự thay đổi của một bit, hai bit hoặc nhiều hơn sẽ dẫn đến sự biến đổi ở đầu ra.
- Nếu hai đầu vào của một S-box bất kì chỉ khác nhau 2 bit ở giữa (bit 3 và 4) thì đầu ra sẽ khác nhau ít nhất 2 bit. Nói cách khác, $S(x)$ và $S(x \text{ XOR } 001100)$ phải khác nhau ít nhất 2 bit.

S1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Hình 1.17: S-box 1

S2	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

Hình 1.18: S-box 2

S3	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

Hình 1.19: S-box 3

S4	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

Hình 1.20: S-box 4

S5	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	0	14	2	13	6	15	0	9	10	4	5	3

Hình 1.21: S-box 5

S6	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

Hình 1.22: S-box 6

S7	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

Hình 1.23: S-box 7

S8	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Hình 1.24: S-box 8

c, Hộp P-box

Mỗi 4 bit đầu ra của các hộp S-box sẽ được ghép lại, theo thứ tự các hộp và được đem vào hộp P-box. Hộp P-Box đơn giản chỉ là hoán vị các bit với nhau.

P =	16	7	20	21
	29	12	28	17
	1	15	23	26
	5	18	31	10
	2	8	24	14
	32	27	3	9
	19	13	30	6
	22	11	4	25

Hình 1.25: P-box

1.2.2. Các khái niệm cơ bản trong mật mã học

Mật mã học sử dụng một số thuật ngữ đặc thù mà người đọc cần nắm rõ trước khi tìm hiểu về DES:

- **Bản rõ (Plaintext):** Là thông tin gốc ở dạng có thể đọc và hiểu được, chưa qua xử lý mã hóa. Đây là dữ liệu đầu vào của quá trình mã hóa.
- **Bản mã (Ciphertext):** Là kết quả sau khi áp dụng thuật toán mã hóa lên bản rõ. Bản mã không thể đọc hiểu được nếu không có khóa giải mã tương ứng.
- **Khóa mã hóa (Key):** Là một tham số bí mật được sử dụng bởi thuật toán mã hóa để biến đổi bản rõ thành bản mã và ngược lại. Độ an toàn của hệ thống mã hóa phụ thuộc trực tiếp vào tính bí mật của khóa.
- **Thuật toán mã hóa (Encryption Algorithm):** Là tập hợp các bước biến đổi toán học được áp dụng lên bản rõ và khóa để tạo ra bản mã.
- **Thuật toán giải mã (Decryption Algorithm):** Là quá trình ngược lại của mã hóa, sử dụng bản mã và khóa để khôi phục lại bản rõ gốc.

1.2.3. Giới thiệu ngôn ngữ lập trình sử dụng trong đề tài

Việc lựa chọn ngôn ngữ lập trình phù hợp đóng vai trò quan trọng trong việc triển khai thuật toán DES, ảnh hưởng trực tiếp đến hiệu năng xử lý, tính dễ đọc của mã nguồn và khả năng mở rộng của chương trình. Trong đề tài này, nhóm sử dụng hai ngôn ngữ lập trình là **Java** và **Python** để cài đặt và so sánh hiệu quả triển khai thuật toán DES.

Tiêu chí lựa chọn hai ngôn ngữ này dựa trên các yếu tố:

- **Hỗ trợ thao tác bit:** Thuật toán DES yêu cầu thực hiện nhiều phép toán ở cấp độ bit như XOR, dịch bit, hoán vị. Cả Java và Python đều hỗ trợ đầy đủ các phép toán này.
- **Khả năng xây dựng giao diện:** Chương trình cần có giao diện trực quan để người dùng dễ dàng nhập dữ liệu, thực hiện mã hóa/giải mã và quan sát kết quả. Java hỗ trợ xây dựng giao diện thông qua thư viện Swing/JavaFX, còn Python có thể sử dụng Tkinter hoặc PyQt.
- **Tính phổ biến và tài liệu hỗ trợ:** Cả hai ngôn ngữ đều có cộng đồng phát triển lớn mạnh, tài liệu tham khảo phong phú và được sử dụng rộng rãi trong lĩnh vực bảo mật thông tin.



Hình 1.26: Java và Python

Việc sử dụng đồng thời cả Java và Python trong đề tài không chỉ cho phép nhóm so sánh cách tiếp cận cài đặt thuật toán DES từ hai góc độ khác nhau, mà còn giúp đánh giá ưu và nhược điểm của từng ngôn ngữ trong bối cảnh ứng dụng mã hóa thực tế. Phần giới thiệu chi tiết về môi trường phát triển, cách cài đặt và các hàm cụ thể của từng ngôn ngữ sẽ được trình bày đầy đủ tại mục 2.4 trong Chương 2.

1.3. Nội dung nghiên cứu

1.3.1. Lý do chọn đề tài

Trong thời đại số hóa hiện nay, thông tin được trao đổi liên tục qua môi trường mạng, kéo theo đó là nguy cơ rò rỉ, đánh cắp và làm sai lệch dữ liệu ngày càng gia tăng. Bảo mật thông tin vì vậy đã trở thành yêu cầu không thể thiếu trong mọi hệ thống công nghệ hiện đại, và mã hóa dữ liệu chính là công cụ cốt lõi để thực hiện điều đó.

Trong số các thuật toán mã hóa, DES (Data Encryption Standard) là một trong những chuẩn mã hóa đối xứng kinh điển nhất, có ý nghĩa học thuật và lịch sử đặc biệt quan trọng. Mặc dù hiện nay DES đã được thay thế bởi các chuẩn mã hóa hiện đại hơn như AES, thuật toán này vẫn là nền tảng quan trọng để hiểu và tiếp cận các hệ thống mã hóa phức tạp hơn, nhờ cấu trúc rõ ràng, các bước xử lý cụ thể và dễ minh họa.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài "Chuẩn DES và ứng dụng trong mã hóa dữ liệu sử dụng Java và Python" với mong muốn nắm vững kiến thức lý thuyết về mã hóa DES, đồng thời vận dụng kỹ năng lập trình để triển khai thuật toán thực tế, từ đó nâng cao nhận thức về tầm quan trọng của bảo mật thông tin trong môi trường số.

1.3.2. Nội dung nghiên cứu

Đề tài tập trung nghiên cứu các nội dung chính sau:

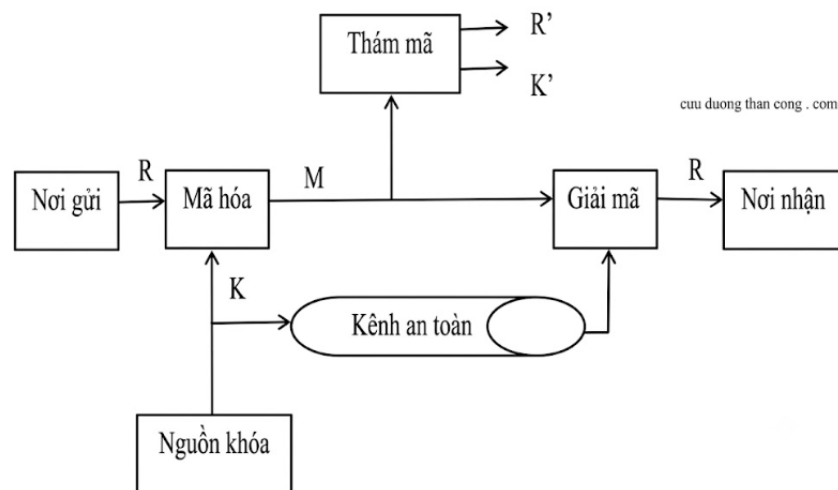
- **An ninh mạng và bảo mật thông tin:** Tìm hiểu các khái niệm cơ bản, nguyên tắc bảo mật và vai trò của mã hóa trong an ninh mạng.
- **Hệ mã hóa khóa bí mật:** Nghiên cứu nguyên lý hoạt động, ưu nhược điểm và các phương pháp mã hóa đối xứng phổ biến.
- **Thuật toán DES:** Nghiên cứu chi tiết cấu trúc và nguyên lý hoạt động của DES, bao gồm:
 - Cấu trúc Feistel và cơ chế 16 vòng lặp.
 - Các bảng hoán vị IP, IP^{-1} , PC1, PC2, E, P.
 - Các hộp thay thế S-box.
 - Quá trình sinh khóa con.
 - Quá trình mã hóa và giải mã dữ liệu.
- **Ứng dụng thực tế của DES:** Tìm hiểu lịch sử hình thành, các lĩnh vực ứng dụng và lý do DES được thay thế bởi các chuẩn mã hóa hiện đại.
- **Cài đặt thuật toán:** Triển khai chương trình mã hóa và giải mã dữ liệu bằng hai ngôn ngữ Java và Python, bao gồm:
 - Xây dựng giao diện người dùng trực quan.
 - Cài đặt các hàm xử lý chính của thuật toán DES.
 - Kiểm thử chương trình với nhiều kịch bản khác nhau.
 - So sánh kết quả triển khai giữa hai ngôn ngữ.

CHƯƠNG 2: KẾT QUẢ NGHIÊN CỨU

2.1. Nghiên cứu, tìm hiểu hệ mã hóa khóa bí mật

2.1.1. Giới thiệu

Mật mã khóa bí mật, hay còn gọi là mật mã đối xứng, ra đời từ rất sớm. Ngay từ trước khi máy tính xuất hiện, mật mã khóa bí mật đã đóng vai trò quan trọng trong việc bảo mật thông tin. Hệ mã hóa này yêu cầu người gửi và người nhận phải thỏa thuận một khóa bí mật trước khi trao đổi thông tin, và khóa đó phải được giữ bí mật tuyệt đối. Độ an toàn của thuật toán phụ thuộc hoàn toàn vào khóa — nếu khóa bị lộ, bất kỳ ai cũng có thể mã hóa và giải mã dữ liệu trong hệ thống.



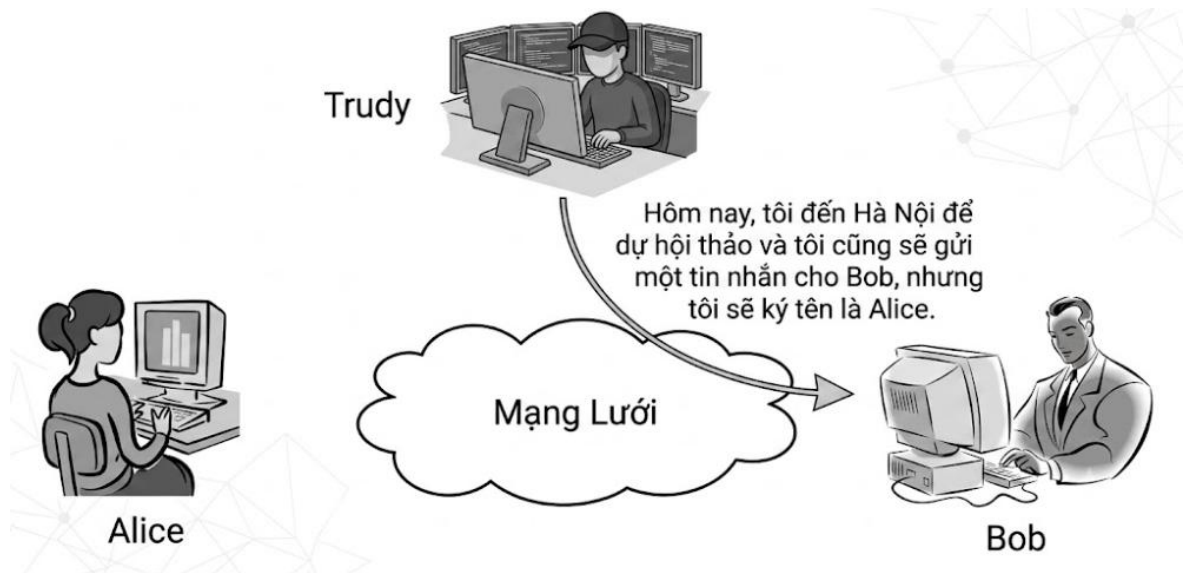
Hình 2.1: Sơ đồ khối của hệ truyền tin mật

Tại nơi gửi, bản rõ R được mã hóa bằng khóa K thông qua thuật toán mã E để tạo ra bản mã $M = E(K, R)$. Tại nơi nhận, với cùng khóa K , thuật toán giải mã D sẽ khôi phục lại bản rõ gốc: $R = D(K, M)$. Nếu kẻ tấn công thu được bản mã M nhưng không có khóa K , họ phải cố gắng khôi phục bản rõ R hoặc tìm ra khóa K . Độ bảo mật của hệ thống chính là thước đo mức độ khó khăn của quá trình này.

2.1.2. Ứng dụng của hệ mã hóa khóa bí mật

a) Tính xác thực

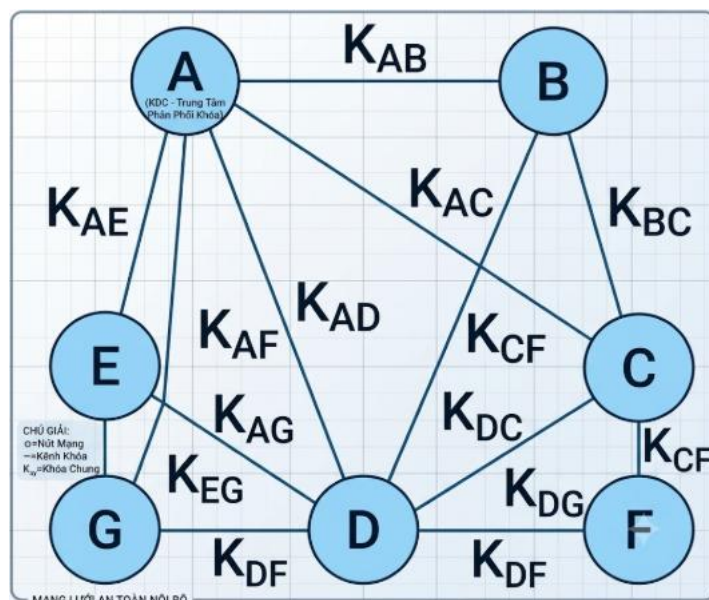
Hệ mã hóa khóa bí mật có thể được dùng để xác thực nguồn gốc thông điệp. Ví dụ, nếu Alice và Bob cùng biết khóa K , Bob có thể xác định thông điệp nào thực sự đến từ Alice dựa vào việc giải mã ra nội dung có nghĩa hay không. Thông điệp giải ra thành chuỗi vô nghĩa có thể kết luận là không phải từ Alice.



Hình 2.2: Ví dụ minh họa tính xác thực trong mã hóa khóa bí mật

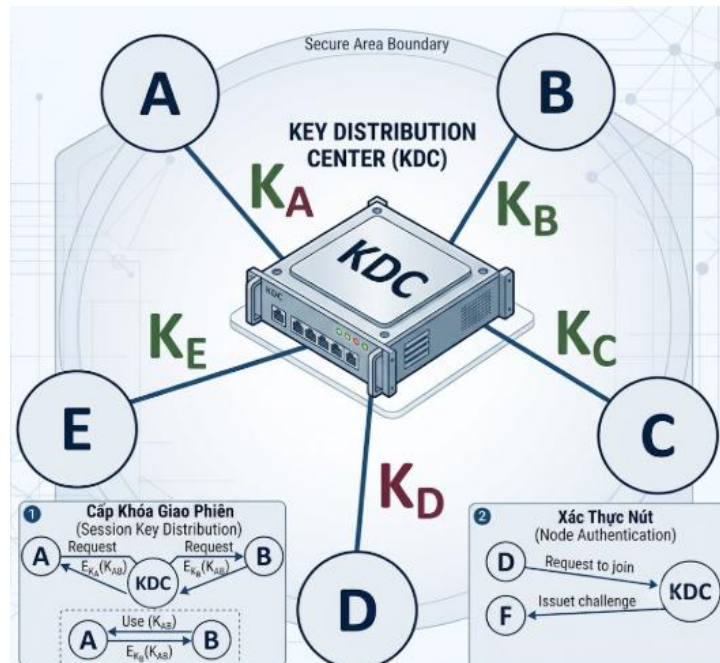
b) Vấn đề trao đổi khóa bí mật

Một trong những nhược điểm lớn nhất của hệ mã hóa đối xứng là bài toán phân phối khóa. Người gửi và người nhận phải thống nhất khóa qua một kênh an toàn trước khi liên lạc. Nếu có N người dùng cần liên lạc theo từng cặp, hệ thống cần đến $N(N-1)/2$ khóa bí mật khác nhau — một con số rất lớn khi N tăng lên.



Hình 2.3: Sơ đồ trao đổi khóa bí mật giữa nhiều người dùng

Để giải quyết vấn đề này, người ta sử dụng Trung tâm phân phối khóa (KDC — Key Distribution Center). Mỗi người dùng chỉ cần giữ một khóa bí mật với KDC; khi hai người muốn liên lạc, KDC sẽ cấp phát một khóa phiên tạm thời cho họ.



Hình 2.4: Sơ đồ phân phối khóa qua KDC

2.1.3. Các phương pháp mã hóa khóa bí mật

Hệ mã hóa khóa bí mật bao gồm nhiều phương pháp khác nhau, từ các kỹ thuật mã hóa cổ điển đến các thuật toán hiện đại:

- **Mật mã cổ điển:** Sử dụng các thao tác đơn giản như thay thế hoặc hoán vị ký tự. Dễ thực hiện thủ công nhưng không đảm bảo an toàn trong bối cảnh hiện đại.
- **Mã dịch chuyển (Caesar):** Mỗi ký tự trong bản rõ được dịch chuyển một số lượng ký tự nhất định trong bảng chữ cái. Ví dụ dịch 3 ký tự: A→D, B→E.
- **Mã thay thế:** Mỗi ký tự được thay bằng một ký tự khác theo bảng quy tắc cố định.
- **Mã hoán vị:** Vị trí các ký tự bị thay đổi theo quy luật nhất định, nhưng giá trị ký tự không đổi.
- **Mã Vigenere:** Mật mã thay thế đa bảng sử dụng một khóa gồm nhiều ký tự lặp lại. An toàn hơn Caesar vì tránh được lặp mẫu thống kê.
- **Mã dòng (Stream Cipher):** Mã hóa từng bit hoặc từng byte liên tiếp. Phù hợp với truyền dữ liệu thời gian thực. Ví dụ điển hình: RC4, ChaCha20.
- **Mã khối (Block Cipher):** Chia bản rõ thành các khối có độ dài cố định và mã hóa từng khối riêng biệt bằng cùng một khóa. Các thuật toán tiêu

biểu: **DES, 3DES, AES, Blowfish**. Đây là loại mã hóa mà nhóm tập trung nghiên cứu trong đề tài này.

2.1.4. Ưu điểm và nhược điểm

Ưu điểm:

- Tốc độ mã hóa và giải mã nhanh, phù hợp với xử lý khối lượng dữ liệu lớn.
- Cài đặt đơn giản, tiêu tốn ít tài nguyên tính toán hơn so với mã hóa bất đối xứng.

Nhược điểm:

- Người gửi và người nhận phải thống nhất khóa qua kênh an toàn — điều này khó thực hiện trong môi trường mạng mở.
- Khi có N người dùng, số lượng khóa cần quản lý là $N(N-1)/2$, tăng nhanh và khó kiểm soát.
- Không hỗ trợ tính chống chối bỏ (non-repudiation) vì cả hai bên đều biết khóa, không thể xác định chính xác ai là người tạo ra bản mã.
- Nếu khóa bị lộ, toàn bộ dữ liệu đã mã hóa bằng khóa đó đều có nguy cơ bị giải mã.

2.2. Chuẩn DES và ứng dụng trong thực tế

2.2.1. Lịch sử và giới thiệu chung về DES

Năm 1971, Horst Feistel đề xuất hệ mã Lucifer hoạt động trên các khối 64 bit với kích thước khóa 128 bit, đây là tiền thân trực tiếp của DES. Năm 1975, công ty IBM phát triển thuật toán DES (Data Encryption Standard) dựa trên hệ mã Lucifer. Đến tháng 11/1976, tổ chức Tiêu chuẩn xử lý thông tin liên bang Hoa Kỳ (FIPS) chính thức công bố DES là chuẩn mã hóa dữ liệu quốc gia, được xuất bản trong tài liệu FIPS PUB 46 vào tháng 01/1977.

DES ra đời trong bối cảnh nhu cầu có một chuẩn mật mã chung ngày càng cấp thiết, khi các thuật toán mã hóa cổ điển không còn đảm bảo được tính bảo mật trong thời đại mạng máy tính phát triển. Một chuẩn chung được yêu cầu phải đáp ứng các tiêu chí: bảo mật cao, thuật toán hoàn toàn công khai, dễ triển khai và linh hoạt trong ứng dụng thực tiễn.

2.2.2. Đặc điểm tổng quan của DES

DES là thuật toán mã hóa khối đối xứng với các thông số cơ bản:

- Độ dài khối dữ liệu: 64 bit
- Độ dài khóa hiệu dụng: 56 bit (64 bit tổng, 8 bit dùng kiểm tra chẵn lẻ)
- Số vòng lặp: 16 vòng
- Mã hóa và giải mã dùng chung một khóa
- Được thiết kế để chạy trên phần cứng

2.2.3. Ứng dụng thực tế của DES

Mặc dù hiện nay DES đã không còn được khuyến nghị sử dụng độc lập, thuật toán này từng được ứng dụng rộng rãi trong nhiều lĩnh vực:

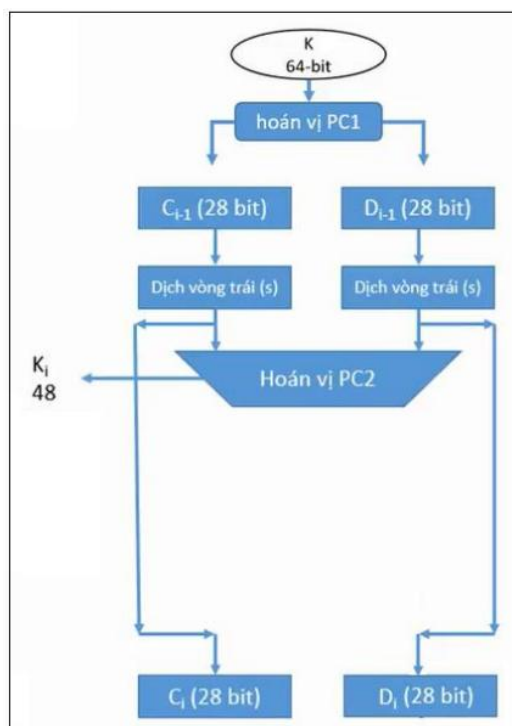
- **Ngân hàng và tài chính:** Mã hóa dữ liệu thẻ và mã PIN trong hệ thống ATM; mã hóa thông điệp giao dịch trong hệ thống chuyển khoản quốc tế SWIFT.
- **Truyền thông bảo mật:** Bảo vệ thông tin liên lạc của các cơ quan chính phủ và quân sự; mã hóa nội dung email và tin nhắn trong các ứng dụng thời kỳ đầu.
- **Mã hóa tệp và lưu trữ:** Tích hợp trong các phần mềm bảo mật để mã hóa tập tin, thư mục và ổ đĩa, đảm bảo dữ liệu không thể đọc được khi bị đánh cắp.
- **Thương mại điện tử:** Mã hóa thông tin thẻ thanh toán và xác thực người dùng trên các nền tảng thương mại điện tử những năm 1990.
- **Quản lý truy cập:** Mã hóa mật khẩu trong cơ sở dữ liệu hệ thống; tích hợp trong hệ thống khóa điện tử và thẻ thông minh để kiểm soát quyền ra vào.

2.3. Nghiên cứu, tìm hiểu về chuẩn DES

2.3.1. Quá trình sinh khóa con

a, Tổng quan

Từ khóa gốc 64 bit, thuật toán DES sinh ra 16 khóa con K_i , mỗi khóa dài 48 bit, tương ứng với 16 vòng Feistel. Sơ đồ tổng quát quá trình sinh khóa con được mô tả trong hình dưới đây.



Hình 2.5: Sơ đồ quá trình sinh khóa con

Quá trình sinh khóa gồm 2 bước chính:

b, Bước 1 – Tính hoán vị PC1

Khóa gốc K (64 bit) được áp dụng hoán vị PC1 để loại bỏ 8 bit kiểm tra chẵn lẻ (các vị trí chia hết cho 8), thu được chuỗi PC1K = 56 bit.

Bảng 2.1: Bảng Hoán vị PC1

K =	1	2	3	4	5	6	7	8	PC1 =	57	49	41	33	25	17	9	1
	9	10	11	12	13	14	15	16		58	50	42	34	26	18	10	2
	17	18	19	20	21	22	23	24		59	51	43	35	27	19	11	3
	25	26	27	28	29	30	31	32		60	52	44	36	28	20	12	4
	33	34	35	36	37	38	39	40		31	23	15	7	62	54	46	38
	41	42	43	44	45	46	47	48		30	22	14	6	61	53	45	37
	49	50	51	52	53	54	55	56		29	21	13	5	28	20	12	4
	57	58	59	60	61	62	63	64									

Chuỗi 56 bit sau PC1 được chia thành hai nửa:

Co: 28 bit đầu (nửa trái)

Do: 28 bit cuối (nửa phải)

Ví dụ minh họa với khóa K = 133457799BBCDFF1 (hệ 16):

Chuyển sang nhị phân: K = 00010011 00110100 01010111 01111001 10011011
10111100 11011111 11110001

Sau khi áp dụng PC1 (bỏ các bit tại vị trí 8, 16, 24, 32, 40, 48, 56, 64):

PC1K = 11110000 11001100 10101010 11110101 01010110 01100111
10001111 = F0 CC AA F5 56 67 8F (hệ 16)

Chia đôi: $C_0 = F0\ CC\ AA\ F$ (28 bit đầu); $D_0 = 5\ 56\ 67\ 8F$ (28 bit cuối)

c) Bước 2 – Sinh các khóa con K_i từ PC1K

Bước 2a – Dịch vòng trái (RotLeftShift)

Với mỗi vòng i ($i = 1 \rightarrow 16$), hai nửa C_{i-1} và D_{i-1} được dịch vòng trái độc lập một số bit S_i theo bảng sau:

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	Tổng
S_i	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1	28

Công thức: $C_i = \text{RotLeftShift}(C_{i-1}, S_i)$ và $D_i = \text{RotLeftShift}(D_{i-1}, S_i)$

Ví dụ vòng 1 (dịch trái 1 bit):

$C_1 = E1\ 99\ 55\ F$ (= C_0 dịch trái 1 bit)

$D_1 = A\ AC\ CF\ 1E$ (= D_0 dịch trái 1 bit)

Bảng 2.2: Bảng giá trị C_i , D_i qua 16 vòng

i	S_i	PC1K	
		C_i	D_i
0		1111 0000 1100 1100 1010 1010 1111	0101 0101 0110 0110 0111 1000 1111
1	1	1110 0001 1001 1001 0101 0101 1111	1010 1010 1100 1100 1111 0001 1110
2	1	1100 0011 0011 0010 1010 1011 1111	0101 0101 1001 1001 1110 0011 1101
3	2	0000 1100 1100 1010 1010 1111 1111	0101 0110 0110 0111 1000 1111 0101
4	2	0011 0011 0010 1010 1011 1111 1100	0101 1001 1001 1110 0011 1101 0101
5	2	1100 1100 1010 1010 1111 1111 0000	0110 0110 0111 1000 1111 0101 0101
6	2	0011 0010 1010 1011 1111 1100 0011	1001 1001 1110 0011 1101 0101 0101
7	2	1100 1010 1010 1111 1111 0000 1100	0110 0111 1000 1111 0101 0101 0110
8	2	0010 1010 1011 1111 1100 0011 0011	1001 1110 0011 1101 0101 0101 1001
9	1	0101 0101 0111 1111 1000 0110 0110	0011 1100 0111 1010 1010 1011 0011
10	2	0101 0101 1111 1110 0001 1001 1001	1111 0001 1110 1010 1010 1100 1100
11	2	0101 0111 1111 1000 0110 0110 0101	1100 0111 1010 1010 1011 0011 0011
12	2	0101 1111 1110 0001 1001 1001 0101	0001 1110 1010 1010 1100 1100 1111
13	2	0111 1111 1000 0110 0110 0101 0101	0111 1010 1010 1011 0011 0011 1100
14	2	1111 1110 0001 1001 1001 0101 0101	1110 1010 1010 1100 1100 1111 0001
15	2	1111 1000 0110 0110 0101 0101 0111	1010 1010 1011 0011 0011 1100 0111
16	1	1111 0000 1100 1100 1010 1010 1111	0101 0101 0110 0110 0111 1000 1111

Bước 2b – Hoán vị PC2

Ghép $C_i D_i$ (56 bit) rồi áp dụng bảng PC2 để chọn ra 48 bit tạo thành khóa con:

$$K_i = \text{PC2}(C_i D_i)$$

Bảng 2.3: Bảng hoán vị PC2

CD =	1	2	3	4	5	6	7	PC2 =	14	17	11	24	1	5	Có 48 bit
	8	9	10	11	12	13	14		3	28	15	6	21	10	
	15	16	17	18	19	20	21		23	19	12	4	26	8	
	22	23	24	25	26	27	28		16	7	27	20	13	2	
	29	30	31	32	33	34	35		41	52	31	37	47	55	
	36	37	38	39	40	41	42		30	40	51	45	33	48	
	43	44	45	46	47	48	49		44	49	39	56	34	53	
	50	51	52	53	54	55	56		46	42	50	36	29	32	

Ví dụ tính K_1 :

$$K_1 = PC2(C_1D_1) = 1B\ 02\ EF\ FC\ 70\ 72$$

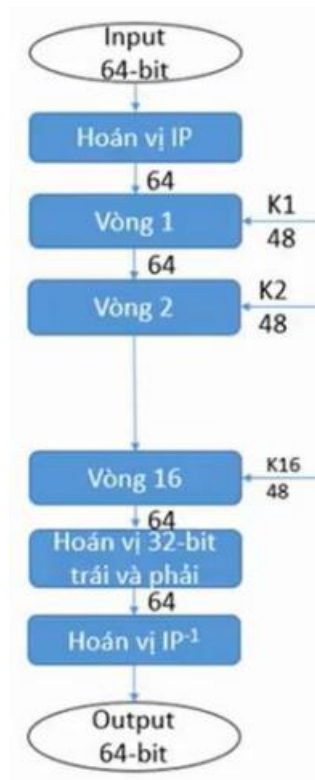
Kết quả 16 khóa con sinh ra từ khóa $K = 133457799BBCDF1$:

i	K_i (hex)
1	1B02EFFC7072
2	79AED9DBC9E5
3	55FC8A42CF99
4	72ADD6DB351D
5	7CEC07EB53A8
6	63A53E507B2F
7	EC84B7F618BC
8	F78A3AC13BFB
9	E0DBEBEDE781
10	B1F347BA464F
11	215FD3DED386
12	7571F59467E9
13	97C5D1FABA41
14	5F43B7F2E73A
15	BF918D3D3F0A
16	CB3D8B0E17F5

2.3.2. Quá trình mã hóa DES

a) Tổng quan

DES mã hóa từng khối bản rõ 64 bit thành bản mã 64 bit bằng khóa 56 bit hiệu dụng, thông qua 4 giai đoạn: Hoán vị IP $\rightarrow$ 16 vòng Feistel $\rightarrow$ Hoán đổi nửa $\rightarrow$ Hoán vị IP^{-1} .



Hình 2.6: Sơ đồ mã hóa DES tổng quát

b) Bước 1 – Hoán vị khởi tạo IP

64 bit bản rõ M được hoán vị theo bảng IP. Sau đó chia thành:

Lo: 32 bit đầu

Ro: 32 bit cuối

Bảng 2.4: Bảng hoán vị IP

M =	1	2	3	4	5	6	7	8	IP =	58	50	42	34	26	18	10	2
	9	10	11	12	13	14	15	16		60	52	44	36	28	20	12	4
	17	18	19	20	21	22	23	24		62	54	46	38	30	22	14	6
	25	26	27	28	29	30	31	32		64	56	48	40	32	24	16	8
	33	34	35	36	37	38	39	40		57	49	41	33	25	17	9	1
	41	42	43	44	45	46	47	48		59	51	43	35	27	19	11	3
	49	50	51	52	53	54	55	56		61	53	45	37	29	21	13	5
	57	58	59	60	61	62	63	64		63	55	47	39	31	23	15	7

Ví dụ với M = 0123456789ABCDEF (hệ 16):

Chuyển sang nhị phân: M = 00000001 00100011 01000101 01100111
10001001 10101011 11001101 11101111

Sau hoán vị IP: IP(M) = 1100 1100 0000 0000 1100 1100 1111 1111 1111 0000
1010 1010 1111 0000 1010 1010 = CC00CCFF0AAF0AA (hệ 16)

Chia hai nửa:

Lo = CC00CCFF (32 bit đầu); Ro = F0AAF0AA (32 bit cuối)

c) Bước 2 – Thực hiện 16 vòng Feistel

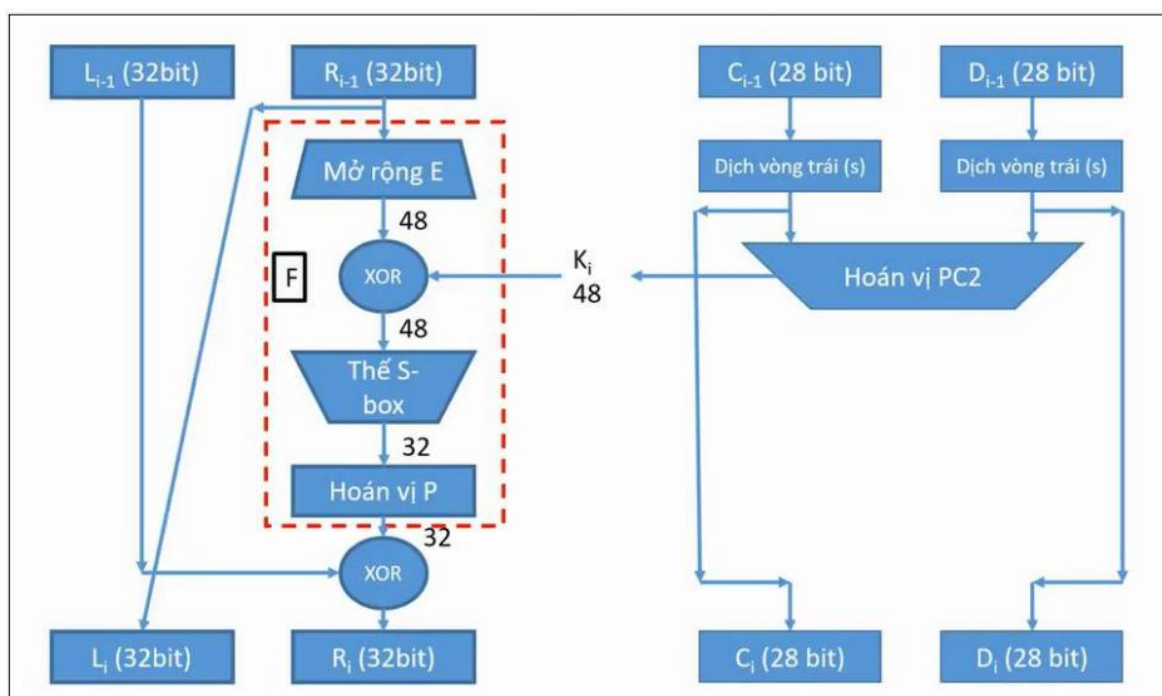
Mỗi vòng i áp dụng công thức:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Bảng 2.5: Bảng 16 vòng lặp L_i, R_i

i	IP(M)	
	L_i	R_i
0	CC00CCFF	F0AAF0AA
1	$L_1 = R_0$	$R_1 = L_0 \oplus f(R_0, K_1)$
2	$L_2 = R_1$	$R_2 = L_1 \oplus f(R_1, K_2)$
3	$L_3 = R_2$	$R_3 = L_2 \oplus f(R_2, K_3)$
4	$L_4 = R_3$	$R_4 = L_3 \oplus f(R_3, K_4)$
	...	...
16	$L_{16} = R_{15}$	$R_{16} = L_{15} \oplus f(R_{15}, K_{16})$



Hình 2.7: Chi tiết một vòng lặp DES

Chi tiết vòng 1 (ví dụ minh họa):

Bước 1 – Mở rộng $E(R_0)$: Áp dụng bảng mở rộng E lên R_0 (32 bit) $\rightarrow$ 48 bit:

Bảng 2.6: Bảng mở rộng E

E =	32	1	2	3	4	5
	4	5	6	7	8	9
	8	9	10	11	12	13
	12	13	14	15	16	17
	16	17	18	19	20	21
	20	21	22	23	24	25
	24	25	26	27	28	29
	28	29	30	31	32	1

$R_0 = 1111\ 0000\ 1010\ 1010\ 1111\ 0000\ 1010\ 1010$ (32 bit)

$E(R_0) = 011110\ 100001\ 010101\ 010101\ 011110\ 100001\ 010101\ 010101 = 7A$

15 55 7A 15 55 (hệ 16)

Bước 2 – XOR với khóa con K_1 :

$K_1 = 000110\ 110000\ 001011\ 101111\ 111111\ 000111\ 000001\ 110010$

$E(R_0) = 011110\ 100001\ 010101\ 010101\ 011110\ 100001\ 010101\ 010101$

$A = E(R_0) \oplus K_1 = 011000\ 010001\ 011110\ 111010\ 100001\ 100110\ 010100\ 100111 = 61\ 17\ BA\ 86\ 65\ 27$ (hệ 16)

Bước 3 – Thay thế S-box:

Chia A thành 8 khối 6 bit: $B_1 = 011000$, $B_2 = 010001$, ..., $B_8 = 100111$

Với mỗi khối $B_i = b_1b_2b_3b_4b_5b_6$: hàng $r = b_1b_6$, cột $c = b_2b_3b_4b_5$

Ví dụ tra S_1 với $B_1 = 011000$:

$b_1b_6 = 00 \rightarrow$ hàng $r = 0$

$b_2b_3b_4b_5 = 1100 \rightarrow$ cột $c = 12$

Tra $S_1[0][12] = 5 \rightarrow$ biểu diễn 4 bit: **0101**

Kết quả sau toàn bộ 8 S-box:

$B = 011000\ 010001\ 011110\ 111010\ 100001\ 100110\ 010100\ 100111$

$S(B) = 0101\ 1100\ 1000\ 0010\ 1011\ 0101\ 1001\ 0111 = 5C\ 82\ B5\ 97$ (hệ 16)

Bước 4 – Hoán vị P:

Bảng 2.7: Bảng hoán vị P

S =	1	2	3	4	P =	16	7	20	21
	5	6	7	8		29	12	28	17
	9	10	11	12		1	15	23	26
	13	14	15	16		5	18	31	10
	17	18	19	20		2	8	24	14
	21	22	23	24		32	27	3	9
	25	26	27	28		19	13	30	6
	29	30	31	32		22	11	4	25

$F = P(SB) = 23\ 4A\ A9\ BB$

Kết quả vòng 1:

$$L_1 = R_0 = F0 \text{ AA } F0 \text{ AA}$$

$$R_1 = L_0 \oplus F = CC00CCFF \oplus 234AA9BB = EF \text{ 4A } 65 \text{ 44}$$

Lặp lại tương tự cho các vòng 2 đến 16 với các khóa con $K_2, \dots, K_{16}$.

Kết quả sau 16 vòng:

$$L_{16} = 43423234; R_{16} = 0A4CD995$$

d) Bước 3 – Hoán đổi nửa và hoán vị IP^{-1}

Sau vòng 16, đổi vị trí hai nửa: ghép $R_{16}L_{16}$ (R trước, L sau).

Áp dụng bảng hoán vị IP^{-1} :

Bảng 2.8: Bảng hoán vị IP^{-1}

$R_{16}L_{16} =$	1	2	3	4	5	6	7	8	$IP^{-1} =$	40	8	48	16	56	24	64	32
	9	10	11	12	13	14	15	16		39	7	47	15	55	23	63	31
	17	18	19	20	21	22	23	24		38	6	46	14	54	22	62	30
	25	26	27	28	29	30	31	32		37	5	45	13	53	21	61	29
	33	34	35	36	37	38	39	40		36	4	44	12	52	20	60	28
	41	42	43	44	45	46	47	48		35	3	43	11	51	19	59	27
	49	50	51	52	53	54	55	56		34	2	42	10	50	18	58	26
	57	58	59	60	61	62	63	64		33	1	41	9	49	17	57	25

$$C = IP^{-1}(R_{16}L_{16}) = 10000101 \ 11101000 \ 00010011 \ 01010100 \ 00001111 \\ 00001010 \ 10110100 \ 00000101 = 85E813540F0AB405 \text{ (hệ 16)}$$

Vậy bản mã: $C = 85E813540F0AB405$

2.3.3. Quá trình giải mã DES

a) Tổng quan

Một điểm đặc biệt của DES là quá trình giải mã có cùng cấu trúc hoàn toàn với mã hóa, chỉ khác ở thứ tự sử dụng các khóa con: giải mã dùng $K_{16}, K_{15}, \dots, K_1$ thay vì $K_1, K_2, \dots, K_{16}$.

b) Cơ sở toán học

Xét vòng i trong mã hóa:

$$L_i = R_{i-1} \text{ và } R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Suy ngược lại:

$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus F(L_i, K_i)$$

Điều này có thể do tính chất tự nghịch đảo của phép XOR: nếu $A \oplus B = C$ thì $C \oplus B = A$. Vì vậy, nếu đưa bản mã qua 16 vòng Feistel với thứ tự khóa $K_{16} \rightarrow K_1$, kết quả thu được chính là bản rõ gốc.

c) Các bước thực hiện

Bước	Mã hóa	Giải mã
1	Hoán vị IP lên bản rõ M	Hoán vị IP lên bản mã C
2	16 vòng với $K_1, K_2, \dots, K_{16}$	16 vòng với $K_{16}, K_{15}, \dots, K_1$
3	Hoán đổi $R_{16}L_{16}$	Hoán đổi $R_{16}L_{16}$
4	Hoán vị $IP^{-1} \rightarrow$ bản mã C	Hoán vị $IP^{-1} \rightarrow$ bản rõ M

Cụ thể, trong mỗi vòng i của quá trình giải mã, khóa con được dùng là K_{17-i} , tức là:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_{17-i})$$

Vòng giải mã	Khóa con dùng
1	K_{16}
2	K_{15}
3	K_{14}
...	...
16	K_1

d) Ví dụ minh họa

Tiếp theo ví dụ ở mục 2.3.2, thực hiện giải mã bản mã $C = 85E813540F0AB405$ với cùng khóa $K = 133457799BBCDFF1$:

Bước 1: Hoán vị IP lên $C \rightarrow$ thu được L_0', R_0'

Bước 2: Thực hiện 16 vòng Feistel theo thứ tự $K_{16}, K_{15}, \dots, K_1$. Hàm F hoạt động giống hệt mã hóa (mở rộng $E \rightarrow \text{XOR} \rightarrow \text{S-box} \rightarrow \text{P-box}$), chỉ khác khóa con đầu vào.

Bước 3: Hoán đổi và hoán vị $IP^{-1} \rightarrow$ thu lại bản rõ gốc:

$$M = 0123456789ABCDEF$$

Kết quả khớp với bản rõ ban đầu, xác nhận tính đúng đắn của thuật toán DES.

2.4. Thiết kế chương trình, cài đặt thuật toán

2.4.1. Ngôn ngữ lập trình sử dụng

Để triển khai thuật toán DES, nhóm sử dụng hai ngôn ngữ lập trình là Java và Python. Cả hai đều được cài đặt theo hướng tự xây dựng thuật toán từ đầu — không sử dụng thư viện mã hóa có sẵn — nhằm hiểu rõ từng bước xử lý bên trong của DES, đồng thời kết hợp với giao diện đồ họa (GUI) để người dùng có thể tương tác trực quan.

2.4.1.1. Ngôn ngữ lập trình Java

a) Giới thiệu

Java là ngôn ngữ lập trình hướng đối tượng, được Sun Microsystems phát triển và ra mắt năm 1995, hiện do Oracle duy trì. Java hoạt động theo nguyên tắc "*Write Once, Run Anywhere*" — mã nguồn được biên dịch sang bytecode và chạy trên máy ảo Java (JVM), đảm bảo tính di động cao trên mọi nền tảng.



b) Môi trường phát triển

Thành phần	Phiên bản sử dụng
JDK (Java Development Kit)	JDK 17 trở lên
IDE	IntelliJ IDEA / Eclipse / NetBeans
Thư viện GUI	Java Swing

c) Lý do lựa chọn Java

Java được lựa chọn vì những ưu điểm phù hợp với yêu cầu của đề tài:

- **Hỗ trợ đầy đủ thao tác bit:** Java cung cấp các phép toán bitwise (&, |, ^, <<, >>, >>>) và kiểu dữ liệu nguyên thủy (int, long) đủ để thực hiện toàn bộ các phép tính của DES như XOR, dịch bit, hoán vị.
- **Xây dựng GUI với Swing:** Thư viện Java Swing cho phép xây dựng giao diện đồ họa đa nền tảng, cung cấp các thành phần như JTextField, JButton, JTextArea phù hợp để thiết kế màn hình nhập liệu và hiển thị kết quả mã hóa/giải mã.
- **Hướng đối tượng rõ ràng:** Cấu trúc class trong Java cho phép tổ chức thuật toán DES thành các module độc lập (sinh khóa, hàm F, mã hóa, giải mã), dễ bảo trì và mở rộng.
- **Hiệu năng ổn định:** JVM tối ưu hóa bytecode trong quá trình chạy, đảm bảo tốc độ xử lý phù hợp cho ứng dụng mã hóa.
- **Phổ biến trong môi trường học thuật:** Java là ngôn ngữ được giảng dạy rộng rãi, tài liệu tham khảo phong phú, cộng đồng hỗ trợ lớn.

2.4.1.2. Ngôn ngữ lập trình Python

a) Giới thiệu

Python là ngôn ngữ lập trình thông dịch, đa năng, được Guido van Rossum phát triển và ra mắt năm 1991. Python nổi bật với cú pháp đơn giản, gần với ngôn ngữ tự nhiên, giúp rút ngắn đáng kể thời gian phát triển so với các ngôn ngữ biên dịch truyền thống.



b) Môi trường phát triển

Thành phần	Phiên bản sử dụng
Python Interpreter	Python 3.10 trở lên
IDE	PyCharm / VS Code
Thư viện GUI	Tkinter (tích hợp sẵn)

c) Lý do lựa chọn Python

Python được lựa chọn vì phù hợp với mục tiêu triển khai và kiểm thử nhanh:

- **Cú pháp ngắn gọn, dễ đọc:** Python cho phép diễn đạt các bước thuật toán DES một cách tường minh, gần với mô tả toán học, giúp dễ kiểm tra tính đúng đắn của từng bước.
- **Hỗ trợ thao tác bit linh hoạt:** Python hỗ trợ đầy đủ các phép toán bitwise (&, |, ^, <<, >>), xử lý số nguyên có độ lớn tùy ý và thao tác với list/string để biểu diễn chuỗi bit.
- **Xây dựng GUI với Tkinter:** Tkinter là thư viện GUI tích hợp sẵn trong Python, không cần cài đặt thêm, cung cấp các widget như Entry, Button, Text đủ để xây dựng giao diện mã hóa/giải mã.
- **Phát triển nhanh:** Là ngôn ngữ thông dịch, Python cho phép kiểm thử từng hàm riêng lẻ ngay lập tức, rút ngắn chu trình debug.
- **Ứng dụng rộng trong bảo mật:** Python được sử dụng phổ biến trong lĩnh vực an ninh mạng, nghiên cứu mật mã học và kiểm thử bảo mật.

2.4.1.3. So sánh tổng quát

Tiêu chí	Java	Python
Kiểu ngôn ngữ	Biên dịch (bytecode / JVM)	Thông dịch
Cú pháp	Chặt chẽ, tường minh kiểu dữ liệu	Đơn giản, linh hoạt
Hiệu năng	Cao hơn	Thấp hơn một chút
Thư viện GUI	Java Swing	Tkinter

Thao tác bit	Đầy đủ, kiểu nguyên thủy cố định	Đầy đủ, số nguyên tùy độ lớn
Thời gian phát triển	Lâu hơn	Nhanh hơn
Ứng dụng trong bảo mật	Phổ biến (backend, enterprise)	Phổ biến (scripting, research)
Mục đích trong đề tài	Cài đặt chính thức, giao diện Swing	Cài đặt kiểm tra, giao diện Tkinter

Việc sử dụng đồng thời cả hai ngôn ngữ cho phép nhóm so sánh cách tiếp cận cài đặt thuật toán DES từ hai góc độ khác nhau, đánh giá sự khác biệt về cú pháp và hiệu quả triển khai. Kết quả mã hóa từ cả hai chương trình được kiểm chứng chéo với nhau và với ví dụ của thầy, đảm bảo tính chính xác của cài đặt.

2.4.2. Cài đặt chương trình bằng Java

2.4.2.1. Luồng hoạt động của chương trình

Chương trình được chia thành hai phần chính hiển thị song song: Mã hóa (bên trái) và Giải mã (bên phải). Mỗi phần bao gồm các bước thao tác rõ ràng, hỗ trợ người dùng thực hiện từng quá trình độc lập.

a, Luồng hoạt động mã hóa

Bước 1: Nhập văn bản gốc: Người dùng có thể nhập trực tiếp vào ô “Văn bản gốc” hoặc nhấn nút Tải tệp văn bản để nhập nội dung từ một tệp ngoài.

Bước 2: Chọn độ dài khóa: Mặc định là 8 bytes (64-bit) – đúng chuẩn yêu cầu của thuật toán DES.

Bước 3: Nhập khóa mã hóa: Người dùng nhập thủ công vào ô “Nhập khóa”, hoặc có thể nhấn nút Sinh khóa để chương trình tự tạo khóa hợp lệ. Có thể Lưu khóa lại hoặc Tải khóa đã lưu trước đó.

Bước 4: Chọn định dạng đầu ra: Người dùng chọn định dạng mã hóa đầu ra, ví dụ: Base64 (hỗ trợ hiển thị và lưu trữ dễ dàng hơn so với dạng nhị phân thuần).

Bước 5: Thực hiện mã hóa: Nhấn nút Mã hóa để hệ thống sử dụng thuật toán DES tiến hành mã hóa văn bản đầu vào với khóa đã cung cấp. Kết quả sẽ hiển thị trong ô “Kết quả mã hóa”.

Bước 6: Lưu kết quả: Người dùng có thể nhấn Lưu kết quả để lưu nội dung đã mã hóa ra tệp, hoặc sao chép trực tiếp từ ô hiển thị.

Bước 7: Xóa nội dung: Nút Xóa dùng để làm trống toàn bộ các ô văn bản ở phần mã hóa để nhập dữ liệu mới.

b) Luồng hoạt động giải mã

Bước 1: Nhập văn bản đã mã hóa: Người dùng nhập trực tiếp văn bản mã hóa vào ô hoặc nhấn Tải tệp mã hóa để nhập từ tệp.

Bước 2: Chọn độ dài khóa: Giống với phần mã hóa – giữ nguyên 8 bytes (64-bit).

Bước 3: Nhập khóa giải mã: Người dùng nhập chính xác khóa đã dùng để mã hóa. Có thể tải khóa từ tệp (Tải khóa) hoặc dùng khóa đang lưu sẵn (Lưu khóa).

Bước 4: Chọn định dạng đầu vào: Chọn định dạng tương ứng với định dạng đầu ra lúc mã hóa, ví dụ: Base64.

Bước 5: Thực hiện giải mã: Nhấn nút Giải mã, chương trình sẽ dùng thuật toán DES với khóa đã nhập để khôi phục lại văn bản gốc từ dữ liệu mã hóa. Kết quả sẽ hiển thị trong ô “Kết quả giải mã”.

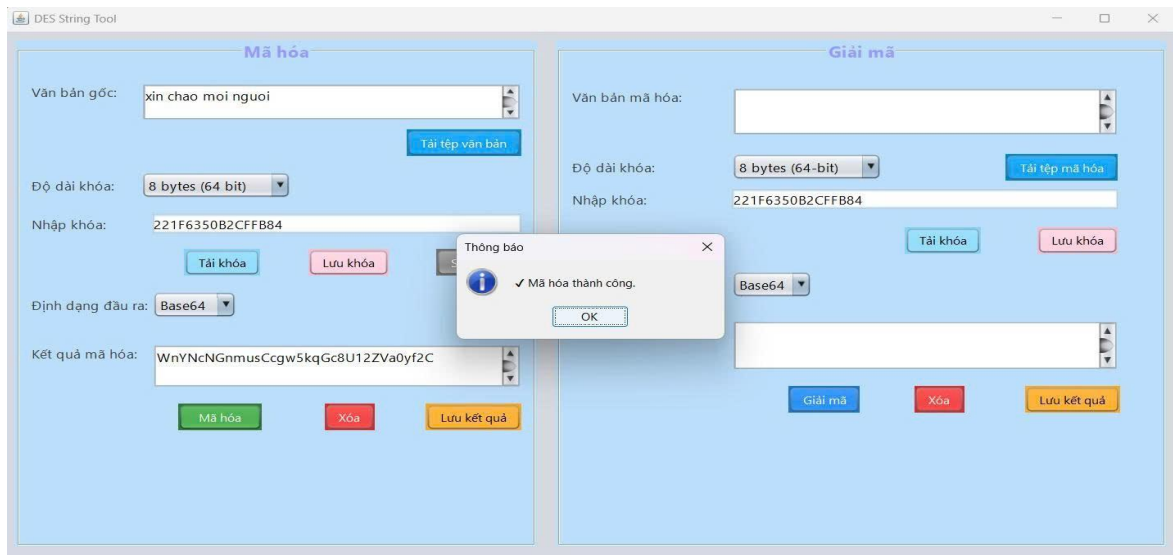
Bước 6: Lưu kết quả: Có thể Lưu kết quả ra tệp hoặc sao chép từ ô kết quả.

Bước 7: Xóa nội dung: Nút Xóa dùng để làm trống toàn bộ các ô ở phần giải mã.

2.4.2.2. Chương trình demo

Bước 1: Thực hiện nhập bản mã và mã hóa

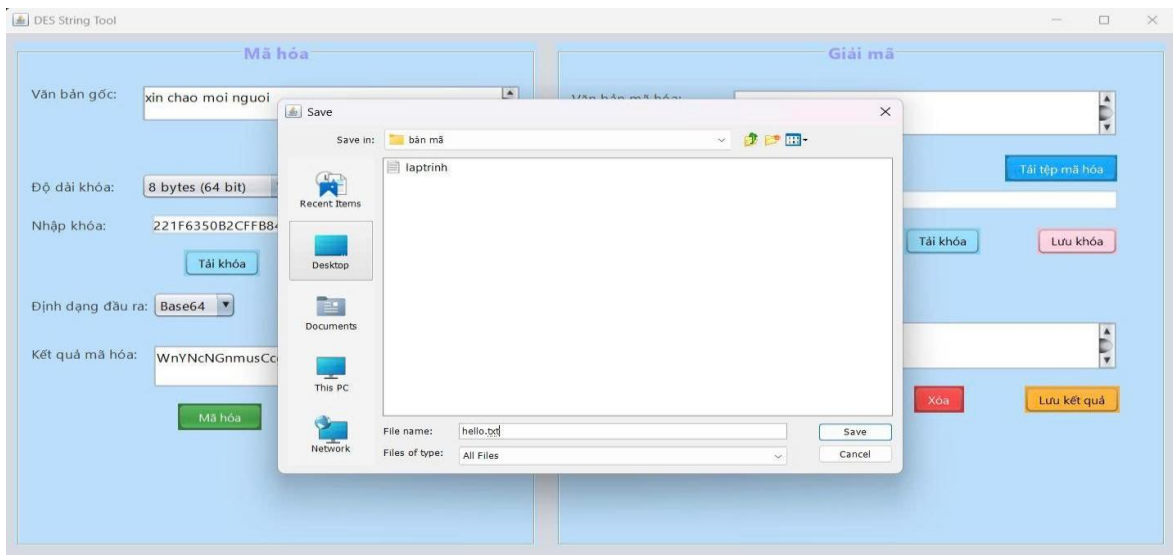
1. Nhập nội dung cần mã hóa vào ô 'Văn bản gốc' (ví dụ: 'xin chào mọi người').
2. Chọn độ dài khóa: 8 bytes (64-bit).
3. Nhập khóa vào ô 'Nhập khóa' (ví dụ: '22F1E653B02CFFB4').
4. Chọn định dạng đầu ra là Base64.
5. Nhấn nút 'Mã hóa' để thực hiện mã hóa.
6. Một thông báo 'Mã hóa thành công' sẽ hiện ra và kết quả hiển thị ở ô 'Kết quả mã hóa'.



Hình 2.8: Minh họa mã hóa thành công

Bước 2: Lưu file bản mã

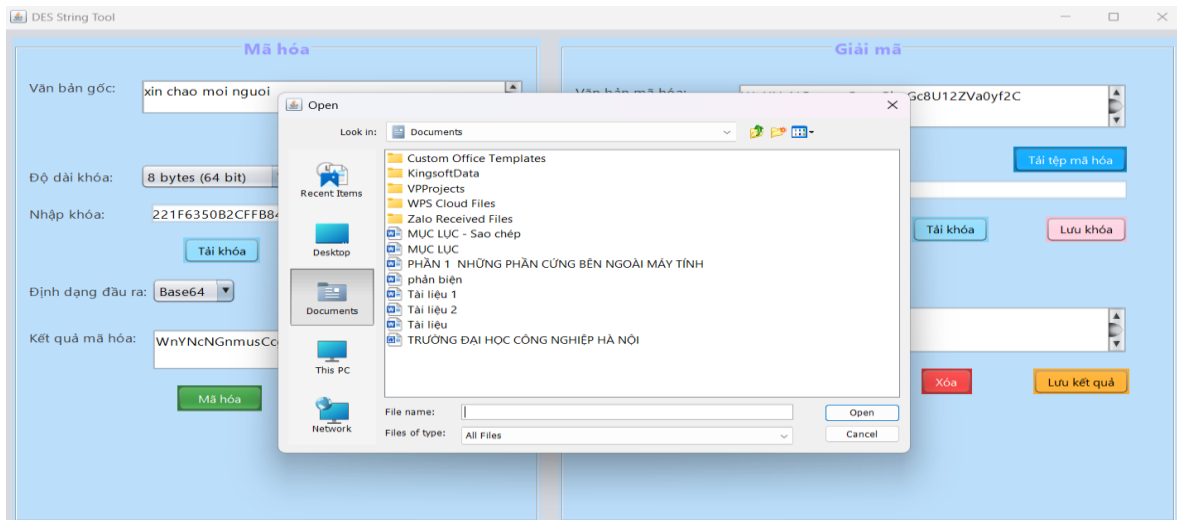
1. Sau khi mã hóa, có thể lưu bản mã bằng cách nhấn nút 'Lưu kết quả'.
2. Hộp thoại lưu file hiện ra, người dùng chọn vị trí và đặt tên file (ví dụ: hello.txt).
3. Nhấn 'Save' để lưu bản mã vào máy tính.



Hình 2.9: Minh họa lưu file bản mã

Bước 3: Tải file bản mã

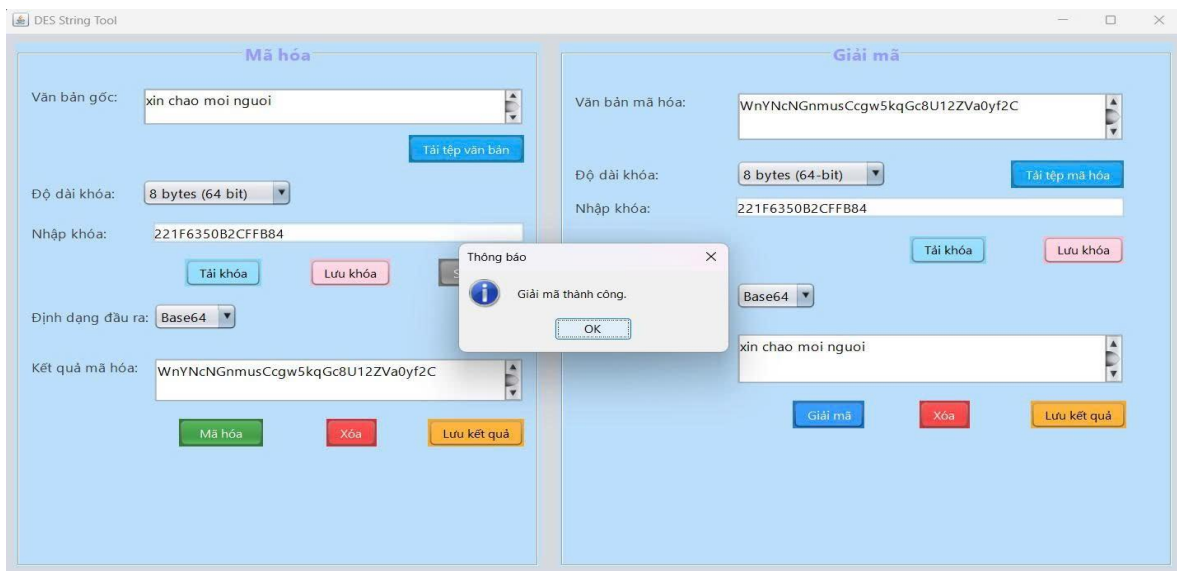
1. Nhấn nút 'Tải tệp mã hóa' ở phần Giải mã.
2. Hộp thoại chọn file xuất hiện, người dùng chọn file chứa bản mã đã lưu.
3. Nhấn 'Open' để tải nội dung bản mã vào chương trình.



Hình 2.10: Minh họa mở file bản mã

Bước 4: Thực hiện giải mã

1. Sau khi tải bản mã và nhập đúng khóa giải mã, nhấn nút 'Giải mã'.
2. Chương trình sẽ hiển thị văn bản gốc đã được giải mã thành công.
3. Một hộp thoại thông báo 'Giải mã thành công' sẽ xuất hiện.



Hình 2.11: Minh họa giải mã thành công

2.4.3. Cài đặt chương trình bằng Python

2.4.3.1. Luồng hoạt động của chương trình

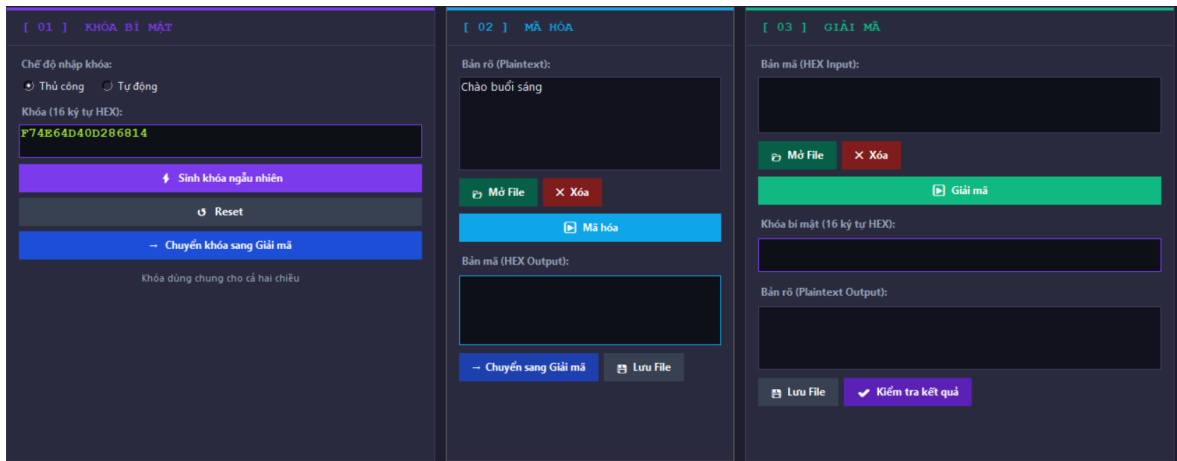
Chương trình được xây dựng bằng Python, kết hợp giữa phần lõi cài đặt thuật toán DES tự xây dựng và giao diện đồ họa Tkinter. Toàn bộ mã nguồn được tổ chức thành hai phần chính:

- **Phần thuật toán** (DES Algorithm Implementation): cài đặt hoàn toàn từ đầu, không sử dụng thư viện mã hóa ngoài.

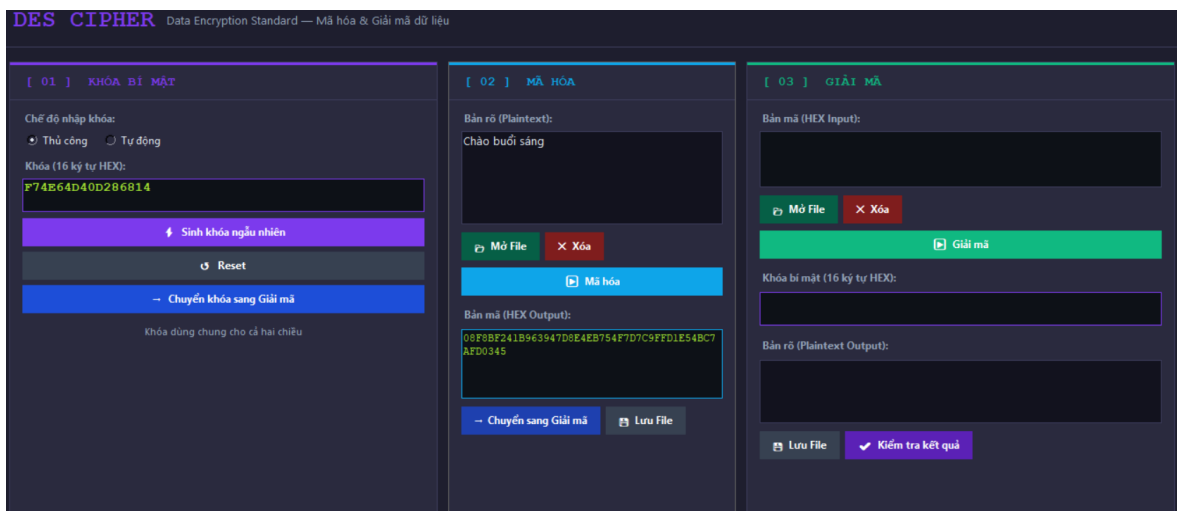
- **Phần giao diện (DESApp):** xây dựng bằng Tkinter, chia thành 3 panel độc lập.

a, Luồng hoạt động mã hóa

1. Sau khi người dùng nhập khóa bí mật(hoặc chọn sinh ngẫu nhiên). Người dùng nhập bản rõ(Ví dụ “Chào buổi sáng”) hoặc có thể nhấn nút “Mở File” để chọn file.txt bản rõ.

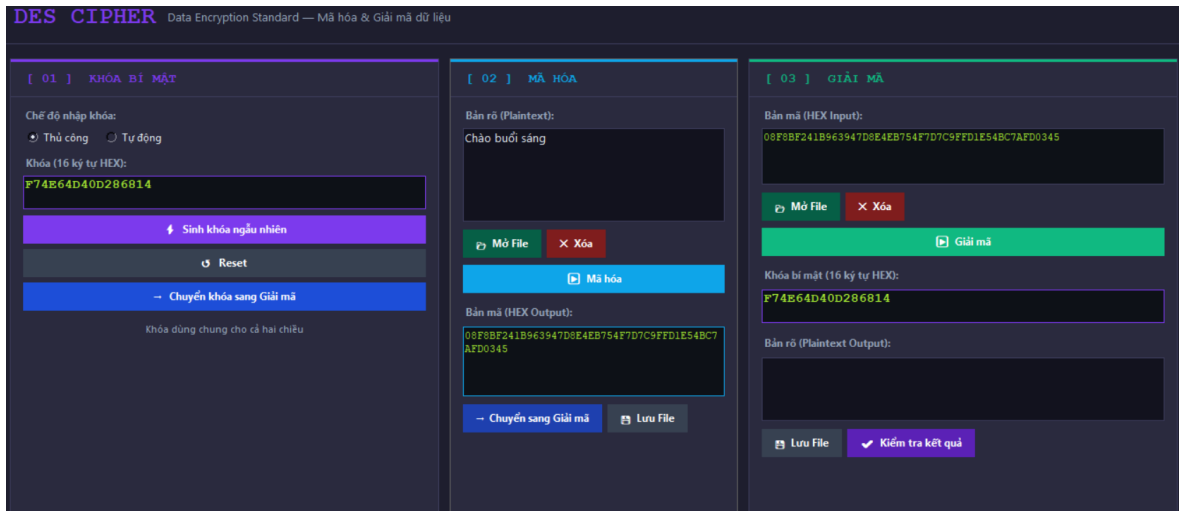


2. Người dùng nhấn nút “Mã hóa” để mã hóa bản rõ thành bản mã.
3. Kết quả bản mã sẽ được hiện ra dưới ô Bản mã. Người dùng có thể lưu file bản mã bằng cách nhấn nút “Lưu File”, hoặc người dùng nhấn nút “Chuyển sang Giải mã” để bắt đầu giải mã.

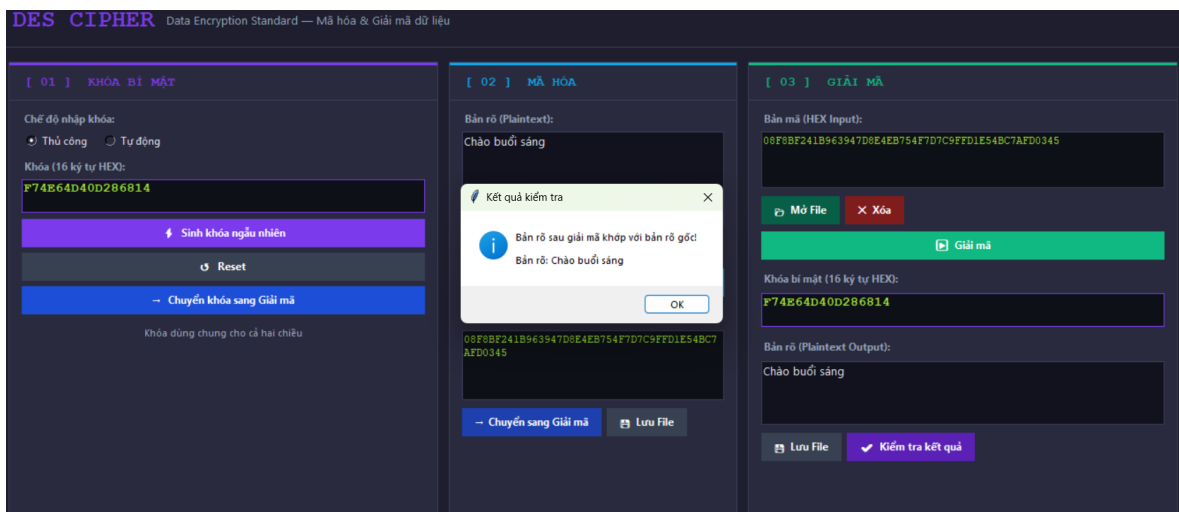


b) Luồng hoạt động giải mã

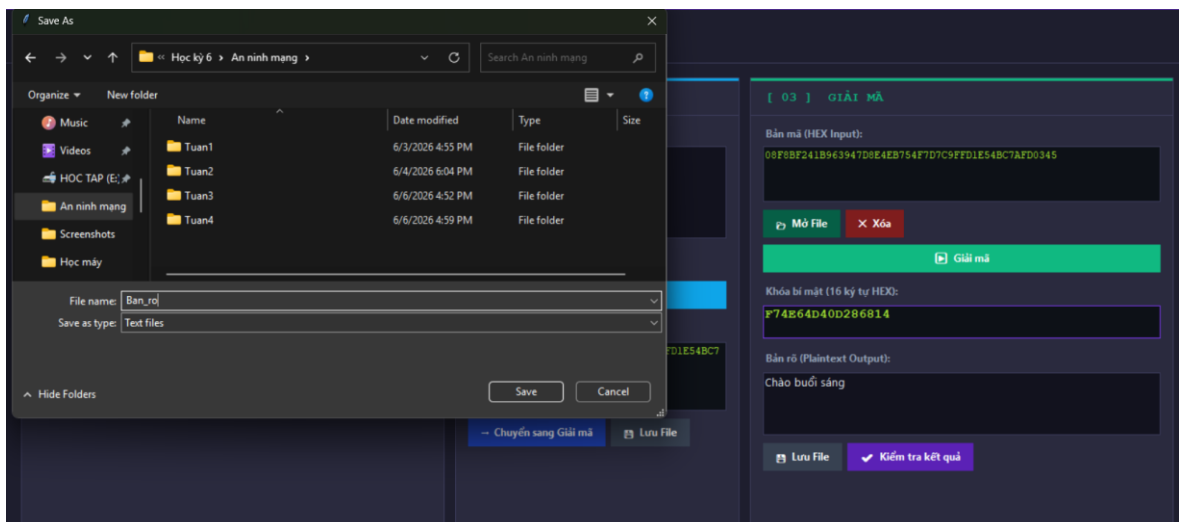
1. Người dùng nhấn nút “Chuyển khóa sang Giải mã” để chuyển khóa bí mật sang vùng giải mã. Người dùng tiếp tục nhập bản mã(hoặc nhấn nút “Chuyển sang Giải mã” để bản mã sang vùng giải mã). Người dùng cũng có thể nhấn nút “Mở File” để chọn file bản mã đã được lưu trước đó.



2. Người dùng nhấn nút “Giải mã” để giải mã bản mã thành bản rõ ban đầu.
3. Kết quả bản rõ sẽ được hiển thị dưới ô Bản rõ. Người dùng có thể nhấn nút “Kiểm tra kết quả” để so 2 bản rõ với nhau.



4. Người dùng có thể lưu file bản rõ bằng cách nhấn nút “Lưu File”.



- c) Các hàm cốt lõi của thuật toán
- Hàm `generate_subkeys()` — Sinh 16 khóa con:

```
def generate_subkeys(key_bits):
    """Generate 16 subkeys from 64-bit key"""
    cd = permute(key_bits, PC1)      # Áp dụng PC1 → 56 bit
    c, d = cd[:28], cd[28:]         # Chia C0 (28 bit) và D0 (28 bit)
    subkeys = []
    for shift in SHIFTS:            # Duyệt 16 vòng
        c = left_shift(c, shift)    # Dịch vòng trái Ci
        d = left_shift(d, shift)    # Dịch vòng trái Di
        subkey = permute(c + d, PC2) # Áp dụng PC2 → 48 bit
        subkeys.append(subkey)
    return subkeys
```

Hàm `f_function()` — Hàm Feistel F:

```
def f_function(right, subkey):
    """Feistel function"""
    expanded = permute(right, E)      # Mở rộng E: 32 → 48 bit
    xored = xor(expanded, subkey)     # XOR với khóa con 48 bit
    sbox_out = []
    for i in range(8):
        block = xored[i*6:(i+1)*6]    # Tách thành 8 khối 6 bit
        row = (block[0] << 1) | block[5] # Hàng: b1b6
        col = bits_to_int(block[1:5])  # Cột: b2b3b4b5
        val = S[i][row][col]           # Tra S-box
        sbox_out += int_to_bits(val, 4) # Kết quả 4 bit
    return permute(sbox_out, P)        # Hoán vị P → 32 bit
```

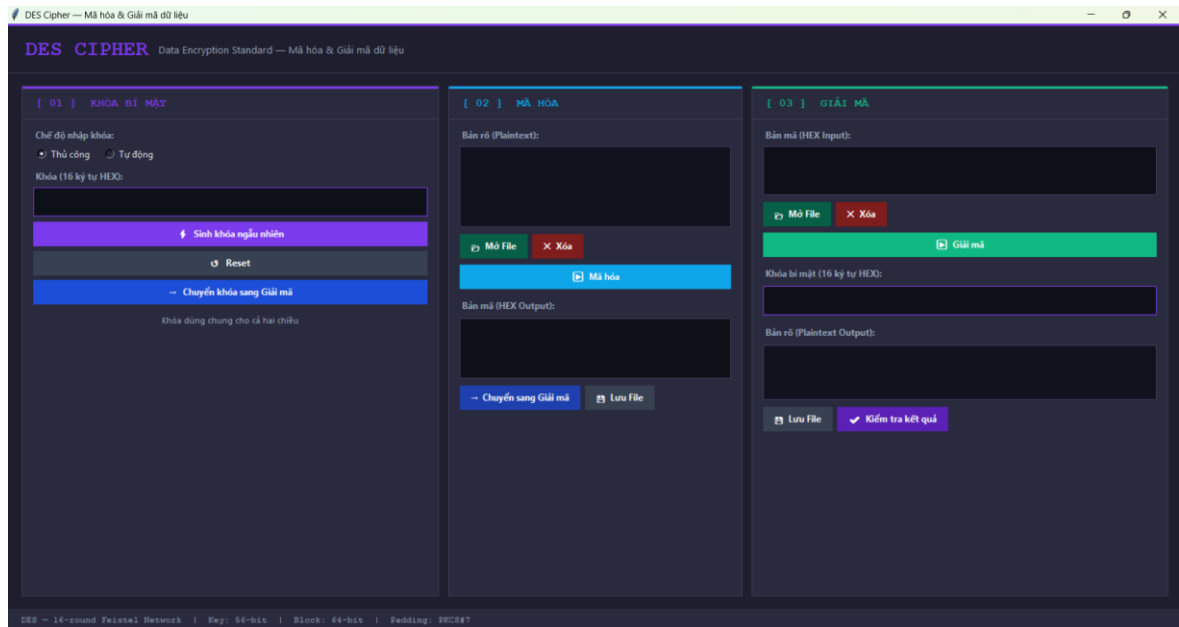
Hàm `des_block()` — Mã hóa/giải mã một khối 64 bit:

```
def des_block(block_bits, subkeys):
    """DES on a single 64-bit block"""
    ip = permute(block_bits, IP)      # Hoán vị IP
    left, right = ip[:32], ip[32:]    # Chia L0, R0
    for subkey in subkeys:            # 16 vòng Feistel
        new_right = xor(left, f_function(right, subkey))
        left = right
        right = new_right
    combined = right + left           # Ghép R16L16 (đổi trái-phải)
    return permute(combined, IP_INV)  # Hoán vị IP-1
```

Hàm `des_block()` dùng chung cho cả mã hóa lẫn giải mã. Sự khác biệt nằm ở thứ tự khóa con: mã hóa truyền `subkeys (K1→K16)`, giải mã truyền `subkeys[::-1] (K16→K1)`.

2.4.3.2. Giao diện chương trình

Giao diện được thiết kế theo bố cục 3 panel ngang, mỗi panel đảm nhận một chức năng riêng biệt:



Hình 2.12: Giao diện tổng thể chương trình DES Python

Panel 1 — Tạo khóa:

- Cho phép người dùng chọn chế độ nhập khóa: Thủ công (tự gõ 16 ký tự HEX) hoặc Tự động (sinh ngẫu nhiên).
- Nút "Sinh khóa ngẫu nhiên": tạo khóa 64-bit ngẫu nhiên dạng HEX.
- Nút "Chuyển khóa": sao chép khóa từ vùng mã hóa sang vùng giải mã.
- Nút "Reset": xóa khóa hiện tại, chuyển về chế độ nhập thủ công.

Panel 2 — Mã hóa:

- Ô nhập Bản rõ: hỗ trợ văn bản UTF-8, có thể nhập trực tiếp hoặc mở từ file .txt.
- Nút "Mã hóa": thực hiện mã hóa DES, hiển thị bản mã dạng HEX ở ô bên dưới.
- Nút "Chuyển sang bản mã": tự động chuyển bản mã và khóa sang Panel 3 để giải mã.
- Nút "Lưu File": lưu bản mã HEX ra file .txt.

Panel 3 — Giải mã:

- Ô nhập Bản mã (HEX): nhập trực tiếp hoặc mở từ file, nhận dữ liệu chuyển từ Panel 2.
- Ô nhập Khóa bí mật: nhận khóa chuyển từ Panel 1/2 hoặc nhập thủ công.
- Nút "Giải mã": thực hiện giải mã, hiển thị bản rõ khôi phục.
- Nút "Kiểm tra kết quả": so sánh bản rõ sau giải mã với bản rõ gốc, thông báo khớp hay không khớp.

- Nút "Lưu File": lưu bản rõ ra file .txt.

2.4.3.3. Kịch bản kiểm thử

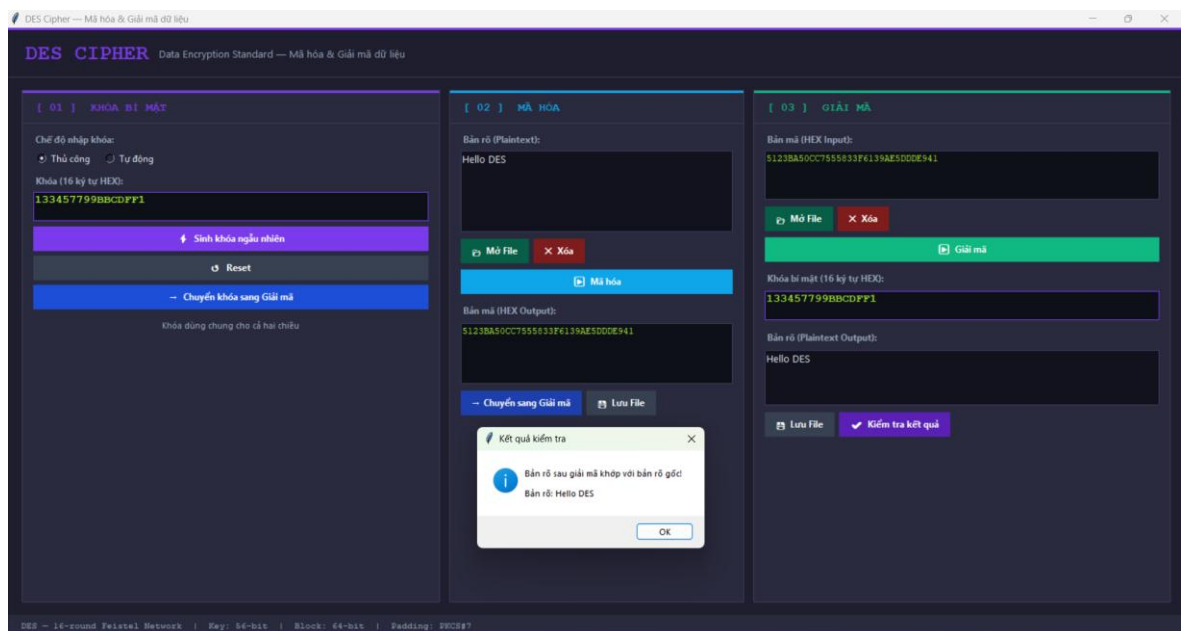
Kịch bản 1 — Mã hóa và giải mã văn bản tiếng Anh

Mục tiêu: Kiểm tra chương trình mã hóa đúng bản rõ tiếng Anh và giải mã trả về đúng bản rõ gốc.

Các bước thực hiện:

1. Tại Panel 1, chọn chế độ **Thủ công**, nhập khóa: 133457799BBCDFF1
2. Tại Panel 2, nhập bản rõ: Hello DES
3. Nhấn nút **"Mã hóa"** → chương trình hiển thị bản mã HEX ở ô bên dưới
4. Nhấn nút **"Chuyển sang bản mã"** → bản mã và khóa tự động chuyển sang Panel 3
5. Tại Panel 3, nhấn nút **"Giải mã"** → bản rõ khôi phục hiển thị
6. Nhấn **"Kiểm tra kết quả"** → thông báo xác nhận kết quả

Kết quả mong đợi: Bản rõ sau giải mã = Hello DES, thông báo "Bản rõ sau giải mã khớp với bản rõ gốc!"



Hình 2.13: Kết quả kịch bản 1

Kịch bản 2 — Mã hóa và giải mã văn bản tiếng Việt có dấu

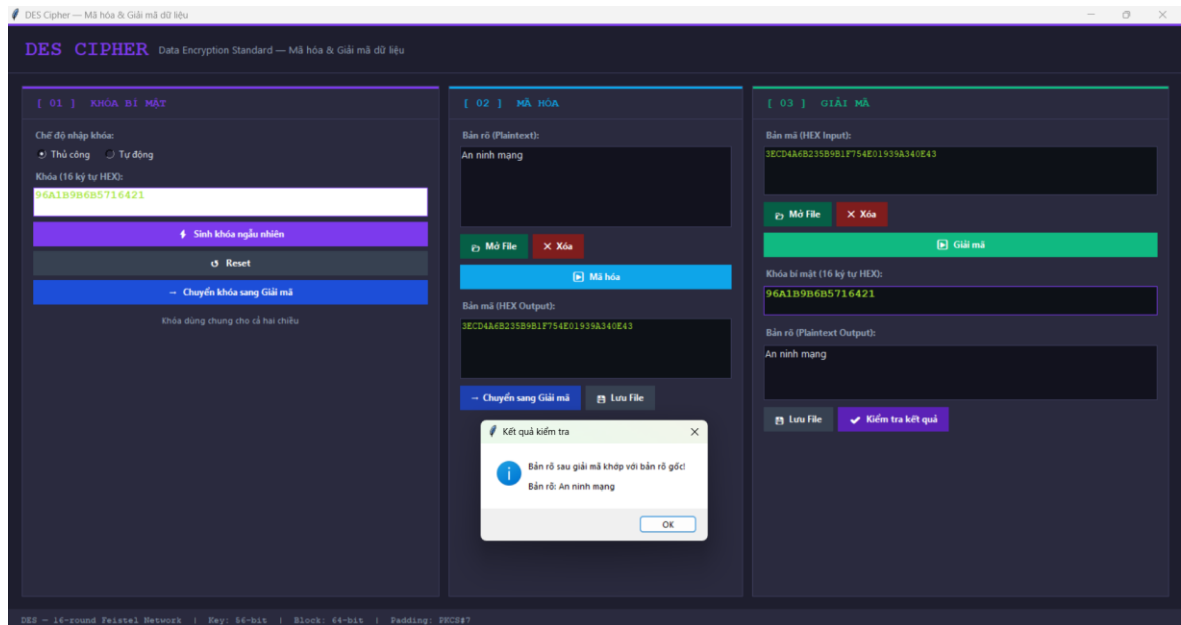
Mục tiêu: Kiểm tra khả năng xử lý ký tự UTF-8 nhiều byte (tiếng Việt).

Các bước thực hiện:

1. Tại Panel 1, chọn chế độ Tự động, nhấn "Sinh khóa ngẫu nhiên" → chương trình tạo khóa ngẫu nhiên
2. Nhấn "Chuyển khóa" sang Panel 3

3. Tại Panel 2, nhập bản rõ: An ninh mạng
4. Nhấn "Mã hóa" → hiển thị bản mã HEX
5. Chuyển bản mã sang Panel 3 và nhấn "Giải mã"
6. Nhấn "Kiểm tra kết quả"

Kết quả mong đợi: Bản rõ khôi phục = An ninh mạng, thông báo khớp.



Hình 2.14: Kết quả kịch bản 2

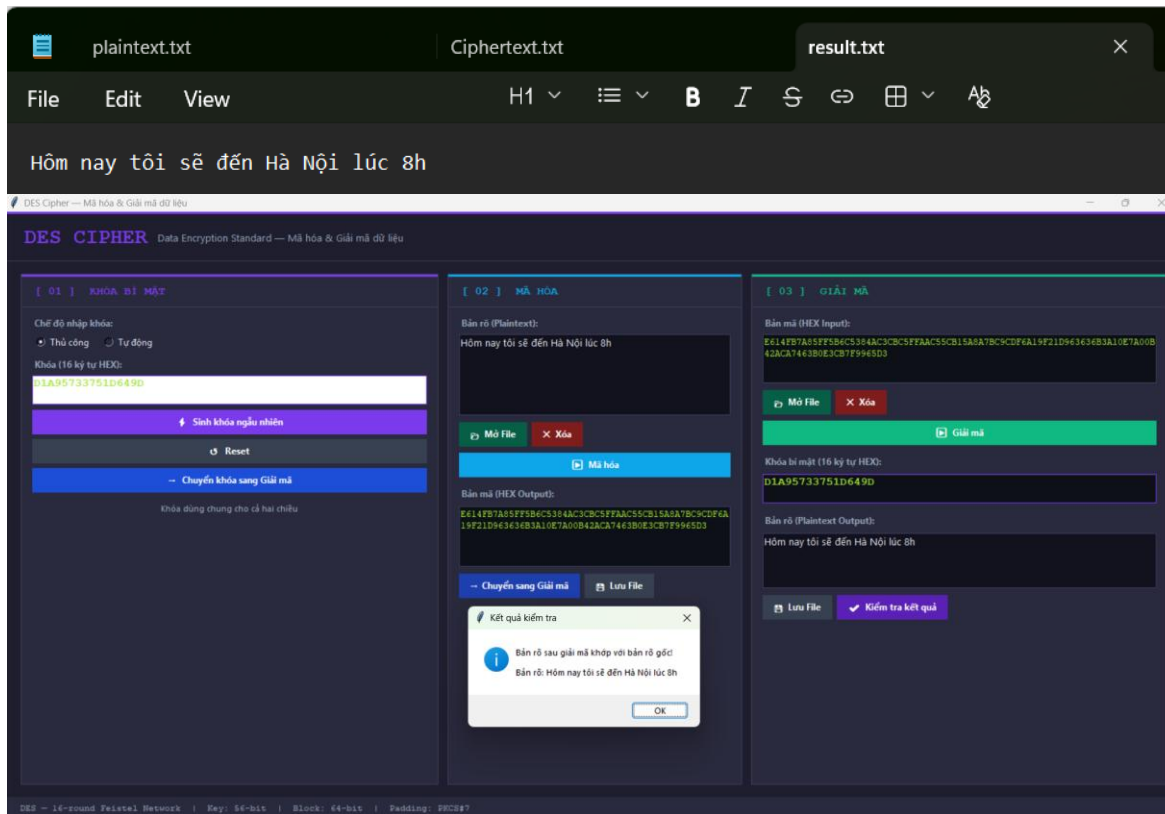
Kịch bản 3 — Giải mã với khóa sai

Mục tiêu: Kiểm tra tính bảo mật — giải mã bằng khóa sai phải cho kết quả sai.

Các bước thực hiện:

1. Mã hóa bản rõ Hello DES với khóa 133457799BBCDFF1 → thu được bản mã HEX
2. Tại Panel 3, nhập bản mã đó nhưng dùng khóa khác: AABBCDDDEEFF0011
3. Nhấn "Giải mã" → chương trình vẫn chạy nhưng cho ra chuỗi ký tự vô nghĩa
4. Nhấn "Kiểm tra kết quả" → thông báo không khớp

Kết quả mong đợi: Bản rõ sau giải mã là chuỗi ký tự lộn xộn, thông báo "Bản rõ sau giải mã không khớp với bản rõ gốc!"



Hình 2.16: Kết quả kịch bản 4

CHƯƠNG 3: KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

3.1. Kiến thức kỹ năng đã học được trong quá trình thực hiện đề tài

Qua quá trình nghiên cứu và thực hiện đề tài "Chuẩn DES và ứng dụng trong mã hóa dữ liệu", nhóm đã tích lũy được nhiều kiến thức và kỹ năng quan trọng trên cả hai phương diện lý thuyết và thực hành.

Về mặt lý thuyết, nhóm đã nắm vững nguyên lý hoạt động của mã hóa đối xứng và cấu trúc mạng Feistel — nền tảng thiết kế của DES. Từ đó, nhóm hiểu rõ từng thành phần trong thuật toán: hoán vị IP và IP^{-1} , bảng mở rộng E, 8 bảng S-box tạo tính phi tuyến, hoán vị P-box, cùng quá trình sinh 16 khóa con qua PC1 và PC2. Bên cạnh đó, nhóm cũng có cái nhìn tổng quan về lịch sử phát triển của DES — từ hệ mã Lucifer (1971), DES chuẩn (1977), đến Triple DES và sự ra đời của AES (2001) khi DES không còn đủ an toàn trước các cuộc tấn công hiện đại.

Về mặt thực hành, nhóm rèn luyện được kỹ năng tự cài đặt thuật toán mật mã từ đầu mà không dựa vào thư viện có sẵn trên cả hai ngôn ngữ Java và Python, đòi hỏi hiểu biết sâu về từng bước tính toán. Qua đó, nhóm thành thạo thao tác bit trong cả hai ngôn ngữ — Python với kiểu dữ liệu linh hoạt và Java với các phép toán bitwise trên kiểu nguyên thủy long — cùng với việc xử lý mã hóa ký tự đa byte UTF-8 để hỗ trợ tiếng Việt. Về giao diện, nhóm tích lũy kinh nghiệm xây dựng GUI bằng Tkinter (Python) và Java Swing (Java), từ thiết kế layout cho đến xử lý sự kiện người dùng và đọc/ghi file. Ngoài ra, nhóm còn rèn luyện kỹ năng làm việc nhóm, phân chia công việc theo từng module, đọc hiểu tài liệu kỹ thuật tiếng Anh và viết báo cáo kỹ thuật có minh họa bằng ví dụ số cụ thể.

3.2. Bài học kinh nghiệm

Trong quá trình thực hiện đề tài, nhóm đã rút ra được một số bài học kinh nghiệm thực tế như sau:

Thứ nhất, việc nắm vững lý thuyết trước khi bắt tay vào cài đặt là yếu tố then chốt. Hiểu rõ từng bước của thuật toán giúp quá trình lập trình diễn ra thuận lợi hơn và hạn chế được các lỗi phát sinh trong quá trình thực hiện.

Thứ hai, cần kiểm thử thường xuyên trong suốt quá trình phát triển thay vì chỉ kiểm tra khi hoàn thiện toàn bộ chương trình. Việc kiểm tra từng phần

nhỏ và đối chiếu với ví dụ minh họa trong tài liệu giúp phát hiện và sửa lỗi nhanh hơn.

Thứ ba, phân chia công việc rõ ràng và hợp lý giữa các thành viên ngay từ đầu giúp nhóm đảm bảo tiến độ, tránh chòng chéo và tổng hợp kết quả cuối cùng thuận lợi hơn.

3.3. Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn

Về tính khả thi:

Chủ đề nghiên cứu về chuẩn DES có tính khả thi cao trong môi trường học thuật. Tài liệu tham khảo phong phú, thuật toán được công bố công khai và có nhiều ví dụ minh họa chi tiết giúp việc nghiên cứu, cài đặt và kiểm chứng kết quả trở nên thuận lợi. Bên cạnh đó, việc tự cài đặt thuật toán từ đầu bằng các ngôn ngữ lập trình phổ biến như Java và Python hoàn toàn nằm trong khả năng của sinh viên ngành Công nghệ Thông tin.

Về thuận lợi:

Nhóm nhận được sự hướng dẫn và hỗ trợ tài liệu đầy đủ từ giảng viên, tạo nền tảng lý thuyết vững chắc để triển khai đề tài. Ngoài ra, các công cụ và môi trường phát triển đều miễn phí và dễ dàng cài đặt, giúp nhóm tập trung vào nội dung chuyên môn thay vì các vấn đề kỹ thuật phụ trợ.

Về khó khăn:

Khó khăn lớn nhất mà nhóm gặp phải là khoảng cách giữa lý thuyết và thực hành. Thuật toán DES có nhiều bảng hoán vị và bước xử lý bit phức tạp, đòi hỏi sự cẩn thận cao trong quá trình cài đặt. Bên cạnh đó, việc đồng bộ tiến độ giữa các thành viên trong nhóm đôi khi gặp khó khăn do lịch học và công việc cá nhân khác nhau.

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Xuân Dũng, *Bảo mật thông tin – Mô hình và ứng dụng*, NXB Thống kê, Hà Nội, 2009.
- [2] Phạm Văn Hiệp, *Bài giảng An ninh mạng – Phần 3: Chuẩn mã dữ liệu DES và AES*, Trường Công nghệ Thông tin và Truyền thông, Đại học Công nghiệp Hà Nội, 2025.
- [3] National Institute of Standards and Technology (NIST), *Data Encryption Standard (DES)*, Federal Information Processing Standards Publication 46-3, U.S. Department of Commerce, Washington D.C., October 1999.
- [4] National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197, U.S. Department of Commerce, Washington D.C., November 2001.
- [5] William Stallings, *Cryptography and Network Security: Principles and Practice*, 7th Edition, Pearson, 2017.
- [6] Whitfield Diffie and Martin Hellman, "New Directions in Cryptography", *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.
- [7] GeeksforGeeks, "Data Encryption Standard (DES) | Set 1", *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/data-encryption-standard-des-set-1/>. [Accessed: 5/2026].
- [8] Python Software Foundation, *tkinter — Python interface to Tcl/Tk*, Python 3 Documentation, [Online]. Available: <https://docs.python.org/3/library/tkinter.html>. [Accessed: 5/2026].
- [9] TutorialsPoint, "Cryptography – DES Algorithm", *TutorialsPoint*, [Online]. Available: https://www.tutorialspoint.com/cryptography/cryptography_des_algorithm.htm. [Accessed: 5/2026].